arXiv:2506.16659v3 [cs.LG] 20 May 2026

Memory-Efficient LLM Pretraining via Minimalist Optimizer Design

Athanasios Glentis [*1] **Jiaxiang Li** [*1] **Andi Han** [2] **Mingyi Hong** [1]

Abstract

Training large language models (LLMs) relies on adaptive optimizers such as Adam, which introduce extra operations and require significantly more memory to maintain first- and second-order moments than SGD. While recent works such as GaLore, Fira and APOLLO have proposed state-compressed memory-efficient variants, a fundamental question remains: *What are the minimum modifications to plain SGD needed to match state-of-the-art pretraining performance?* We systematically investigate this question using a bottom-up approach, and identify two simple yet highly (memory- and compute-) efficient techniques: (1) column-wise gradient normalization (normalizing the gradient along the output dimension), that boosts SGD performance without momentum; and (2) applying first-order momentum only to the output layer, where gradient variance is highest. Combining these two techniques lead to SCALE (Stochastic Column-normAlized Last-layer momEntum), a simple optimizer for memory efficient pretraining. Across multiple models (60M–1B), SCALE matches or exceeds the performance of Adam while using only 35–45% of the total memory. It also consistently outperforms memory-efficient optimizers such as GaLore, Fira and APOLLO, making it a strong candidate for large-scale pretraining under memory constraints. For LLaMA 7B, SCALE outperforms the state-of-the-art memory-efficient methods APOLLO and Muon in both perplexity and memory consumption. Code is available at this link.

1. Introduction

Adaptive optimizers such as RMSProp (Hinton et al., 2012), Adam (Kingma & Ba, 2015) are the default optimizers for large-scale attention-based deep neural networks, particularly large language models (LLMs). While effective, these optimizers incur significant memory overhead due to their reliance on maintaining both first- and second-moment estimates of the gradient, which are also known as *optimizer states*. Specifically, Adam requires storing two additional gradient states per parameter, tripling the memory usage compared to vanilla stochastic gradient descent (SGD).

On the other hand, despite its superior memory efficiency, the vanilla SGD performs poorly when applied directly to LLM training, due to the absence of adaptive scaling in the update step (See Figure 2 for experiment results, also see (Zhao et al., 2025; Zhang et al., 2020b)). This has motivated a wave of recent research focused on developing memory-efficient alternatives to Adam that aim to retain its performance while reducing memory consumption. These include compression-based algorithms such as GaLore (Zhao et al., 2024), Fira (Chen et al., 2024) and APOLLO (Zhu et al., 2025). Novel optimizers, such as Muon (Jordan et al., 2024), Scion (Pethick et al., 2025) and SWAN (Ma et al., 2025), also by design require less memory than Adam. These approaches often introduce new algorithmic components—such as different forms of gradient normalization, whitening and rescaling, momentum variants—and combine them in various ways.

However, despite the growing literature on memory-efficient optimizers, there has been no systematic study to identify which specific algorithmic components or techniques are most essential for designing highly performant yet minimal-memory optimizers. For instance, are both first- and second-order momentum terms strictly necessary for effective training? Among the many forms of gradient normalization, which subset of them strike the best trade-off between performance, memory, and computational cost? In the absence of such a principled investigation, it remains unclear how to balance optimizer complexity against memory savings.

This motivates our central research question:

> Can we design a memory efficient optimizer with minimum modifications to plain SGD that achieves state-of-the-art pretraining performance?

[1]Department of Electrical and Computer Engineering, University of Minnesota, USA. [2]School of Mathematics and Statistics, University of Sydney, Australia. Correspondence to: Athanasios Glentis <glent007@umn.edu>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

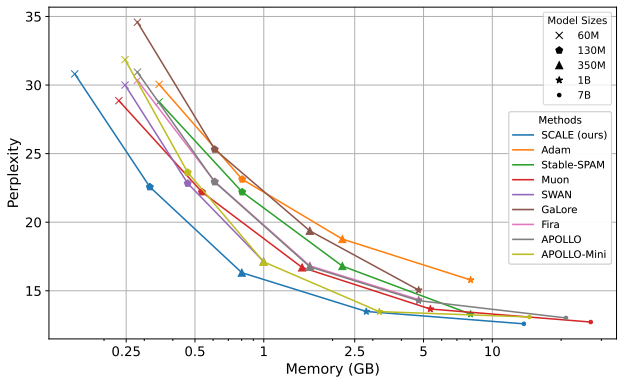


Figure 1. Perplexity v.s. memory consumption among a number of SOTA algorithms. Solutions achieved towards the left-bottom side of the plot represent better performance/memory trade-off (see Appendix B for the details of the memory estimation).

In this work, we address this question through a *bottom-up*, *minimalist* approach. Rather than starting from existing adaptive optimizers, we systematically evaluate the role of fundamental components, namely normalization and momentum, to determine the smallest set of techniques needed to bridge the gap between vanilla SGD and Adam, with a memory efficiency focus. Towards this end, we perform extensive experiments to identify key components (such as different forms of normalization and different levels of momentum) that can effectively enhance the performance of vanilla SGD with minimum memory overhead. We then justify the design choices using a combination of theoretical insights and empirical evidence.

Contributions. Our study suggests that two techniques, when used together, are particularly effective: *(i)* Among various gradient normalization techniques in the literature, column-wise gradient normalization (normalizing along the output dimension) can significantly boost SGD performance, while having simple closed-form solution and with no extra memory required; and *(ii)* adding first-order momentum exclusively to the output layer, where gradient variance is highest (see Figure 4), is surprisingly effective to further boost training performance with minimum memory overhead. Combining these two techniques lead to Stochastic Column-normalized Last-layer momentum (`SCALE`), a memory efficient optimizer which requires roughly the same amount of memory as vanilla SGD. For example, for a 1B (resp. 7B) model, `SCALE` only requires 10% (resp. 2%) more memory as compared to vanilla SGD. Meanwhile, `SCALE` achieves similar performance as compared with SOTA optimizers Adam and Muon, with only 35% and 52% of the memory cost for training 1B models, respectively. Comparing to other memory efficient optimizers, `SCALE` achieves superior performance to GaLore, Fira and Apollo, with only 59% of the memory cost for training 1B models. See Figure 1 for an illustration of performance and memory trade-off among different SOTA algorithms.

1.1. Related Works

Memory-efficient variations of Adam. A recent line of works aims to improve the memory efficiency of Adam by compressing the historical states stored, namely the first- and second-order statistics. Early works include Adafactor (Shazeer & Stern, 2018), which estimates the optimizer states using per-row and per-column moving averages, SM3 (Anil et al., 2019), a memory efficient AdaGrad via grouping second-order momentum, and CAME (Luo et al., 2023), which improves upon Adafactor via matrix factorization. A number of recent works use gradient projections to compress the optimizer states. GaLore (Zhao et al., 2024), being one of the pioneering works, stores the states in low-rank subspaces that captures most gradient information. Fira (Chen et al., 2024) achieves superior performance than GaLore by re-introducing full-rank information to the low-rank gradients. APOLLO (Zhu et al., 2025) constructs the update based on gradient scaling factors that are estimated from the ratio between the low-dimensional gradient and the low-dimensional Adam update; APOLLO-Mini is a memory efficient version of APOLLO by estimating in a rank-1 subspace. GRASS (Muhamed et al., 2024) improves GaLore by designing sparse projection matrices guided by the norm of the rows of the gradients. SlimAdam (Kalra et al., 2025) compresses second-order moments based on signal-to-noise analysis. Some other methods group parameters into blocks and apply block-wise updates (Luo et al., 2024; Ramesh et al., 2024; Pan et al., 2024) or block-wise scaling (Zhang et al., 2024) to further reduce the memory costs.

Towards removing optimizer states. More recent works start to directly remove the optimizer states as required by Adam. Muon (Jordan et al., 2024; Liu et al., 2025) has shown impressive results using only first-order momentum and orthogonalizing it via an iterative algorithm. Scion (Pethick et al., 2025) explores various (layer-wise) normalization schemes for better training performance. SWAN (Ma et al., 2025) and its variant (Scetbon et al., 2025) combine two normalizations (see Section 2 for details) directly to the gradient, matching the performance of Adam (when applied only to the intermediate layers while the first and last layers still use Adam. See Section 4 for details). Zhao et al. (2025) demonstrates SGD with signed momentum is able to give similar performance to Adam. SGD-SaI (Xu et al., 2024) verifies that proper learning-rate scaling at initialization is sufficient to achieve good performance.

2. Methodology

We begin by reviewing several key techniques used in popular optimizers such as Adam and Muon, which have been shown to accelerate pretraining performance for LLMs. This understanding will be critical for our subsequent minimalist design for a memory-efficient optimizer. Denote the

optimization problem of the LLM pretraining as

$$\min_{\theta=[\theta_1,\ldots,\theta_L]} \ell(\theta) := \frac{1}{n}\sum_{i=1}^{n} \ell(\theta;\xi_i) \tag{1}$$

where ℓ is the loss function, usually taken as the cross entropy loss of predicting the next token, $\theta = [\theta_1, ..., \theta_L]$ is the model trainable parameters, with θ_l the l-th layer, $l = 1, 2, ..., L$. With attention-based network, we can simply assume that each $\theta_l \in \mathbb{R}^{d_{l,\text{in}} \times d_{l,\text{out}}}$ is a weight matrix, with the input dimension $d_{l,\text{in}}$ and output dimension $d_{l,\text{out}}$. Here ξ_i with $i = 1, ..., n$ represents training data samples and n is the training data size.

To solve (1), **vanilla SGD** draws a small batch of i.i.d samples $\{\xi_{t,b}\}_{b=1,..,B}$ at iteration t and performs an update toward the negative stochastic gradient direction:

$$\theta^{t+1} = \theta^t - \eta_t g^t, \quad \text{with} \quad g^t := \frac{1}{B}\sum_{b=1}^{B} \nabla \ell(\theta^t;\xi_{t,b}) \tag{2}$$

where B is batch size and η_t is the learning rate. Although memory-efficient, plain SGD performs poorly in LLM training due to the lack of adaptive scaling (Zhao et al., 2025), and we verify this in Figure 2 where we run SGD and Adam on LLaMA 130M pretraining task (see Section 4 for the details of our experiment settings). Therefore we will not include the perplexity result for SGD in the subsequent experiments.

In contrast, adaptive algorithms such as **Adam** (Kingma & Ba, 2015) update parameters using a more sophisticated scheme (where $\odot$ represents element-wise product):

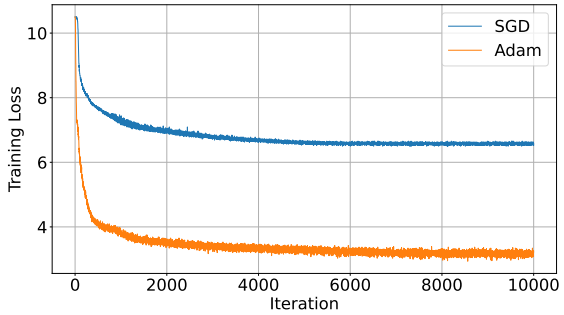
$$\begin{aligned} m^t &= \beta_1 m^{t-1} + (1-\beta_1) g^t, \\ v^t &= \beta_2 v^{t-1} + (1-\beta_2) g^t \odot g^t, \\ \theta^{t+1} &= \theta^t - \eta_t \frac{m^t}{\sqrt{v^t} + \epsilon}. \end{aligned} \tag{3}$$

Numerous theoretical works such as Reddi et al. (2019); Zhang et al. (2022b;a) attempted to explain why Adam outperforms SGD for transformers, though consensus remains elusive. A straightforward decomposition of Adam identifies two essential components.

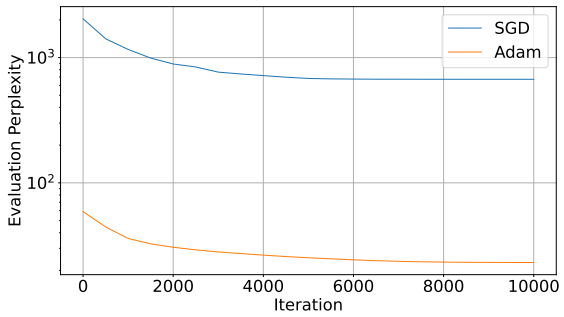
1. **Gradient Normalization**. The key difference between Adam and SGD is the normalization factor v^t in the denominator. One could first normalize each element of SGD, resulting in the sign-SGD update as follows:

$$\theta^{t+1} = \theta^t - \eta_t \frac{g^t}{\sqrt{g^t \odot g^t}} = \theta^t - \eta_t \operatorname{sign}(g^t) \tag{4}$$

where sign stands for taking the sign for each element.



(a) Training Loss



(b) Evaluation Perplexity

Figure 2. Comparison of SGD and Adam training loss and evaluation perplexity curves on LLaMA 130M model. Clearly, SGD is not converging to any reasonable level of perplexity. The Adam and SGD learning rates are 3e-3 and 0.1, respectively. We search with multiple learning rates for SGD; for lower ones the loss decreases even slower and higher ones cause the training to diverge.

2. **Exponential Moving Average (EMA)**. The stochasticity of the mini-batch sample $\{\xi_{t,b}\}_{b=1,..,B}$ of g^t could be smoothed by taking the exponential moving average (EMA) for both the numerator and denominator of the update (4), resulting in the following numerator and denominator of the Adam update:

$$\begin{aligned} m^t &= \beta_1 m^{t-1} + (1-\beta_1) g^t, \\ v^t &= \beta_2 v^{t-1} + (1-\beta_2) g^t \odot g^t. \end{aligned} \tag{5}$$

Combining (4) and (5) results in the original Adam update (3).

This leads to the question: **How much of Adam's benefit arises from gradient normalization versus EMA?** We note that gradient normalization does not require maintaining states, whereas EMA does. Therefore, in the next sections, we will examine each of these components separately and discuss their effectiveness. We will start from the gradient normalization, as the EMA step will introduce extra memory no matter how hard we compress it. Then we will discuss how to use EMA in a more memory-friendly way to further boost the performance of the optimizer.

2.1. Gradient normalization

Gradient normalization is a critical component for large-scale, efficient pretraining, and has remained indispensable

even in recent (near)-stateless optimizers (Ma et al., 2025; Huang et al., 2025; Zhu et al., 2025). It is known to accelerate escape from saddle points (Levy, 2016; Murray et al., 2019), improve the effective smoothness of the objective (when the Hessian norm is upper bounded in terms of the gradient norm) (Zhang et al., 2020a; Kunstner et al., 2023), stabilize gradient distributions (Ma et al., 2025), and provide robustness to heavy-tailed noise (Sun et al., 2025).

Denote $G \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ the stochastic gradient (we switch to upper letters since we assume the weight blocks θ_l in (1) are matrices). There is a variety of gradient normalization schemes, including

$$\begin{aligned}
&\text{Singular-value normalization: } UV^\top,\ G = U\Sigma V^\top \text{ (SVD)} \\
&\text{Column-wise normalization: } \left[\frac{\text{col}_1(G)}{\|\text{col}_1(G)\|}, \ldots, \frac{\text{col}_n(G)}{\|\text{col}_n(G)\|}\right] \\
&\text{Row-wise normalization: } \left[\frac{\text{row}_1(G)^\top}{\|\text{row}_1(G)\|}, \ldots, \frac{\text{row}_m(G)^\top}{\|\text{row}_m(G)\|}\right]^\top \\
&\text{Sign normalization: } \text{sign}(G)
\end{aligned} \tag{6}$$

Here we assume that d_{in} is the input dimension and d_{out} is the output dimension. Therefore row- and column-normalizations correspond to normalizing along the input and output dimensions, respectively. It can be verified that different normalization schemes arise naturally from the steepest ascent direction under different matrix norms (Bernstein & Newhouse, 2024a;b; Pethick et al., 2025; Bernstein & Newhouse, 2025). Many existing optimizers employ different normalization techniques. For example, Sign-SGD/Adam utilize $\|\cdot\|_{1\to\infty}$ (sign normalization)[1] to achieve gradient normalization, Muon (Jordan et al., 2024) utilizes $\|\cdot\|_{2\to 2}$ norm (singular-value normalization), whereas SCION (Pethick et al., 2025) and SlimAdam (Kalra et al., 2025) apply different norms for different layers to achieve better performance. Below we systematically analyze the effect of each normalization scheme separately.

Computational cost of different normalizations. In terms of the computational cost, the normalization techniques discussed in (6) can be quite different. In particular, singular-value normalization requires computing full SVD, which is the most time consuming compared to the rest. Even though efficient approximation methods of SVD have been studied in Jordan et al. (2024); Ma et al. (2025) (e.g., the Newton-Schulz (NS) procedure), they are still much more time consuming as compared with the other three normalization techniques, see Table 1 for a test on the time required for different normalizations.

dimension d	1024	2048	4096
singular-value	79.77	354.27	1958.66
singular-value (NS)	6.03	7.00	14.41
column-wise	0.10	0.12	0.17
row-wise	0.09	0.11	0.13
sign	0.03	0.03	0.03

Table 1. Time (ms) consumed by each of the normalization methods on a torch matrix tensor with dimension $d_{\text{in}} = d_{\text{out}} = d$, testing on a single NVIDIA A40 GPU. Here the singular-value normalization are computed both exactly (first row, using `torch.linalg.svd` directly) and inexactly using Newton-Schulz (NS) iteration (second row, see (Jordan et al., 2024) for details)[2].

	60M	**130M**	**350M**
Tokens	1.4B	2.6B	7.8B
Adam	30.05	23.13	18.77
Adam (Stable-SPAM)	28.77	22.20	16.80
singular-value (NS)	34.15	25.25	18.73
column-wise	39.89	28.85	20.38
row-wise	79.27	37.67	21.63
sign	54.36	40.42	27.95

Table 2. Results (perplexity) of pretraining LLaMA models on C4 dataset, using different gradient normalizations, as specified in (6). For the singular-value normalization, we use the inexact Newton-Schulz (NS) iteration (again see (Jordan et al., 2024) for details) for fast approximation.

Experimental comparison of SGD with different gradient normalizations. We conduct the preliminary experiment of pretraining LLaMA models (See Section 4 for the experiment setting) using SGD with different normalizations applied to all the layers as specified in (6). We report the pretraining perplexities in Table 2, and we notice that all the normalizations improve over SGD, however none of them alone could match the performance of Stable-SPAM (Huang et al., 2025) (which is a stabilized version of Adam). In particular, the four normalization methods can be categorized into two groups based on their performance: singular-value and column-wise normalization achieve better results than row-wise and sign normalization. We thus proceed with the group of normalizations with better performance, namely the singular-value and column-wise normalizations.

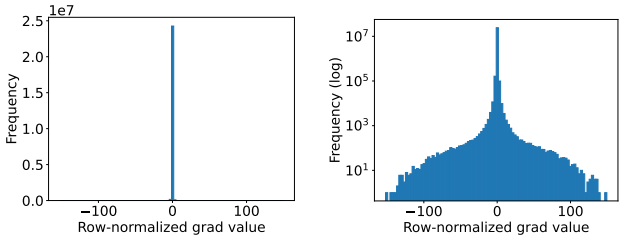
Why row-wise normalization performs poorly. Table 2 shows that row-wise gradient normalization consistently achieves worse performance than column-wise. We identified that this is largely attributed to the effect of row-wise normalization to the last layer's (LM-head) gradients. Figure 3 shows that after row-wise is applied to the LM-head, some gradients obtain very high absolute values (up-to 150 in this case), which destabilizes the training. In contrast, column-wise results into a gradient distribution without ex-

[1] Here we use the matrix induced norm. For any matrix $A \in \mathbb{R}^{n\times m}$, define

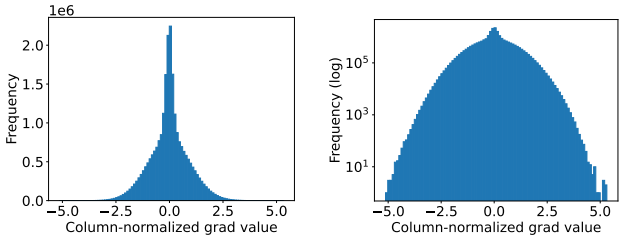
$$\|A\|_{a\to b} = \max_{x\in\mathbb{R}^m} \frac{\|Ax\|_b}{\|x\|_a}$$

where $\|\cdot\|_a$ and $\|\cdot\|_b$ are vector ℓ_a and ℓ_b norms.

[2] The small time difference of column- and row-wise normalization may originate from the way PyTorch stores tensors with strides.



(a) Row-wise normalized LM-head gradients



(b) Column-wise normalized LM-head gradients

Figure 3. The histograms of the LM-head gradients after applying row-wise (a) and column-wise normalization (b). The gradients are from the 1000th training iteration of a LLaMA 130M model. It can be seen from figure (a) that row-wise results into some very high gradient values (up-to range 150 in absolute value) that we find to destabilize training.

treme values, thus aiding stable training.

We believe that this behavior is largely related to the role of the LM-head, i.e., the layer with gradient matrix $d_{\text{model}} \times |V|$ that maps the last hidden state (of dimension d_{model}) to the vocabulary logits (of dimension $|V|$ being the vocabulary size). In particular we have observed that for certain very frequent tokens, their corresponding LM-head column gradients have much larger norms compared to the rest, which if not normalized appropriately (i.e., using column-wise normalization) can lead to unstable training. We provide further discussion and results about the importance of column-wise for the LM-head in Appendix M.

2.2. Momentum in the last layer

First-order momentum (m^t in (3)) has been shown to effectively reduce the variance toward the true gradients; see Liu et al. (2020); Cutkosky & Mehta (2020). Although second-order momentum is used in adaptive optimizers such as Adam and RMSprop, more recent optimizers such as Muon have demonstrated remarkable success without second-order momentum. It is worth noticing that momentum is the major factor that introduces memory overhead in optimizer states, therefore we only consider first-order momentum in our optimizer design. Based on a minimalist optimizer design principle, we plan to investigate whether first-order momentum can be *selectively* used among different layers, while still leading to performance improvements without significantly increasing memory overhead.

Layer-wise gradient variance. Existing works already show the effectiveness of momentum in reducing variances (Liu et al., 2020; Cutkosky & Mehta, 2020). Naturally, if the gradients for different layers have different variances, we could achieve both memory efficiency and variance reduction by applying momentum to the layers with large variances. In the next paragraphs, we identify the layer-wise gradient variances and determine the most important layer that provides the largest performance gain when incorporating momentum; we also justify the effectiveness of different momentum parameters for different layers using a convergence analysis theory in Theorem 2.1.

We first perform a simple experiment on LLaMA 130M to check the gradient variance of different layers. To estimate the gradient variance, one needs to obtain the full gradient (by using the entire training dataset) which is not feasible in practice. Instead, we take a much larger training data batch as input[3] to estimate the true gradient. The experiment results of running SGD with column-wise normalization (SGD-col-norm) and SGD-col-norm with last layer momentum are shown in Figure 4. Interestingly, we observe that the variance of the last layer is the largest among all the layers during training, necessitating a specific treatment for the last layer to reduce the variance.

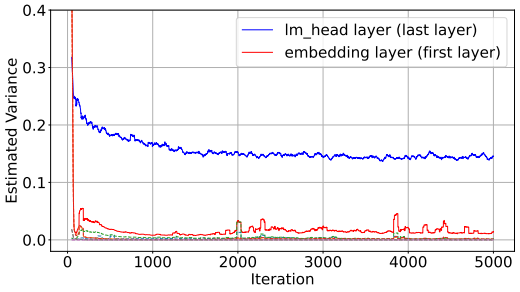
Next, we show theoretically that momentum helps the most for the layers with larger gradient variances. We inspect the theoretical property of applying SGD with momentum (SGD-M) to the LLM optimization problem (1). Note that here we did not consider column-wise normalization for ease of theoretical analysis. Consider the following SGD-M algorithm to solve (1) ($m_l^0 = 0$ and θ^1 is randomly initialized):

$$\begin{aligned} &m_l^t = \beta_l m_l^{t-1} + (1-\beta_l) g^t,\ g^t = \nabla_{\theta_l} \ell(\theta^t; \xi_t) \\ &\theta_l^{t+1} = \theta_l^t - \eta_l m_l^t \end{aligned} \tag{7}$$

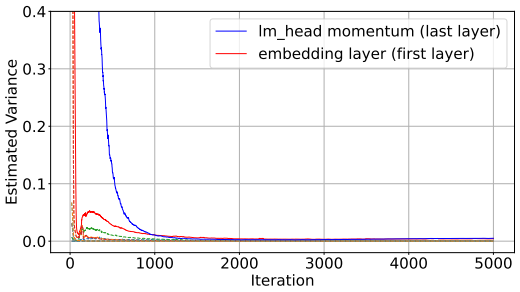
where $t = 1, 2, ..., T$ is the iteration counter; $l = 1, ..., L$ represents different layers, i.e. we assume different layers contain different momentum with different hyperparameters. We have the following theoretical result (see Appendix N for the proof).

Theorem 2.1. *Suppose $\ell(\theta)$ in (1) is lower bounded by ℓ^*, γ-smooth (i.e. $\nabla \ell(\theta)$ is Lipschitz continuous with constant γ), also the stochastic gradient is unbiased $\mathbb{E}_{\xi_t} \nabla_{\theta_l} \ell(\theta^t; \xi_t) = \nabla_{\theta_l} \ell(\theta^t)$ and with bounded variance: $\mathbb{E}_{\xi_t} \|\nabla_{\theta_l} \ell(\theta^t; \xi_t) - \nabla_{\theta_l} \ell(\theta^t)\|^2 \le \sigma_l^2$ for all $l = 1, ..., L$ and $t = 0, ..., T-1$. With appropriate choice of hyperparameters η_l (defined in (34)) and $\beta_l \le 1 - \delta$ (δ is an absolute constant), we have*

[3] For example, we could take the small training batch size as 32 and the large batch size as 512.



(a) SGD-col-norm



(b) SGD-col-norm-mmt-last

Figure 4. Estimated variance of the stochastic gradients (and momentum when applicable) for different layers in two methods (smoothed by 50 iterations window). We observe that when running SGD with column-wise normalization (SGD-col-norm, left plot), the variance of the last layer (lm_head) is largest for most of the time, followed by the variance of the first layer (embedding) and other layers. After applying momentum to the last layer (SGD-col-norm-mmt-last, right plot), the variance of the momentum of last layer (lm_head momentum) decreases to a very low level. Interestingly, the variance of the first layer in plot (b) is also smaller than the one in plot (a).

the following convergence result for update (7):

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{l=1}^{L}\mathbb{E}\|\nabla_l^t\|^2 \le \frac{2L\gamma^{3/2}\mathbb{E}\Delta_1}{\delta^2\sqrt{T}} + \sum_{l=1}^{L}\left(\frac{1-\beta_l}{1+\beta_l}\frac{L\sqrt{\gamma}}{4\sqrt{T}} + \frac{L\gamma^{3/2}}{2\sqrt{T}} + \frac{1-\beta_l}{\beta_l^3}\frac{\gamma^2}{4LT}\right)\frac{\sigma_l^2}{\delta^2} \tag{8}$$

where $\nabla_l^t = \nabla_{\theta_l}\ell(\theta^t;\xi_t)$ *and* $\Delta_1 = \ell(\theta^1) - \ell^*$.

Remark 2.2. The left-hand side of (8) is the expectation of the stochastic gradient, measuring the stationarity. The right-hand side suggests an overall $\mathcal{O}(1/\sqrt{T})$ rate of convergence in terms of the iteration number T, meanwhile the summation indicates the error to each of the layer is aggregated, whose magnitude depends on the variance level σ_l of the specific layer.

Theorem 2.1 suggests that by taking different β_l for different layers $l = 1, ..., L$, the convergence could be improved. The second term of the right-hand side of (8) takes the form

$$f(x) = \frac{1-x}{1+x} + c\frac{1-x}{x^3},\ c := \frac{\gamma}{L^2T^{3/2}}$$

where x represents the layer-wise momentum hyperparameter β_l. It is straightforward to verify that $f(x)$ is decreasing in $(0, 1-\delta]$, therefore an optimal strategy is to pick $\beta_l = 1-\delta$. However from a memory-efficient point of view, only $\beta_l = 0$ saves the memory of momentums.

Now, if σ_l (variance of the gradient of the l-th layer) is significantly higher than other layers, then taking β_l higher than other layers will result in better convergence, which is consistent with our experimental findings. On the other hand, if σ_l is close to) zero, taking $\beta_l = 0$ will provide memory-efficiency without harming the convergence too much. In particular, if the variances $\sigma_l \approx 0$ for $l = 1, ..., L-1$, one could take $\beta_l = 0$ for $l = 1, ..., L-1$ and only keep the momentum for the last layer without significantly damaging the convergence rate.

Proof Sketch for Theorem 2.1. The proof follows Liu et al. (2020), with the major difference being that Liu et al. (2020) proved the case with one single layer ($L = 1$) and we extend it to multi-layer setting ($L > 1$) where each layer has different variance level.

First, define the auxiliary sequence by

$$z_l^t := \begin{cases} \theta_l^t & t = 1 \\ \frac{1}{1-\beta_l}\theta_l^t - \frac{\beta_l}{1-\beta_l}\theta_l^{t-1} & t \ge 2 \end{cases} \tag{9}$$

we can verify two important properties: $z_l^{t+1} - z_l^t = -\eta_l g_l^t$ and $z_l^t - \theta_l^t = -\frac{\beta_l}{1-\beta_l}\eta_l m_l^{t-1}$ (Lemma N.3). Using γ-Lipschitz smooth, we can expand the function value difference via

$$\mathbb{E}_{\xi_t}\left[\ell\left(z^{t+1}\right)\right] \le \ell\left(z^t\right) + \underbrace{\mathbb{E}_{\xi_t}\left[\left\langle\nabla\ell\left(z^t\right), z^{t+1} - z^t\right\rangle\right]}_{(a)} + \frac{\gamma}{2}\underbrace{\mathbb{E}_{\xi_t}\left[\left\|z^{t+1} - z^t\right\|^2\right]}_{(b)}.$$

Expanding the term (a) on the right-hand side using the update using the above properties of z_l^t, we will arrive at a $-\mathcal{O}(\sum_{t=1}^{T}\eta_t\sum_{l=1}^{L}\mathbb{E}\|\nabla_l^t\|^2)$ term (which is the left-hand side of (8)), and some addition term. The additional term, combining with (b), will be bounded via Lemma N.1 and N.2, and together form the right hand side of (8).

Experiments on different gradient normalization with last-layer-momentum. It is observed in Table 2 that singular-value and column-wise normalization obtains better performance than the other normalizations for pretraining. We conduct another experiment to check the performance of the two types of normalization with last-layer-momentum. The results are summarized in Table 3, where we observe that as the model size gets larger, both singular-value and column-wise normalizations + last-layer-momentum are matching Adam's performance, and we choose column-wise due to the computational time concern in Table 1.

Model Size	60M	130M	350M
Tokens	1.4B	2.6B	7.8B
Adam	30.05	23.13	18.77
Adam (Stable-SPAM)	28.77	22.20	16.80
Singular-val (NS) + mmt-last	31.20	22.33	16.67
Column-wise + mmt-last (ours)	30.81	22.57	16.32

Table 3. Evaluation perplexity of two normalizations (singular-value and column-wise) when combined with last-layer-momentum (mmt-last).

Algorithm 1 SCALE: Stochastic Column-normalized Last-layer Momentum

Input: Initialized trainable parameters θ^0, hyperparameters β_t and $\eta_{t,l}$.
for $t = 0, 1, ..., T-1$ **do**
  Sample mini-batch data $\{\xi_{t,b}\}_{b=1,...,B}$;
  for Layers $l = 1, ..., L$ **do**
    Compute the stochastic gradient as $g_l^t := \frac{1}{B}\sum_{b=1}^{B} \nabla_{\theta_l}\ell(\theta^t; \xi_{t,b})$;
    if $l = L$ (last layer) **then**
      $m_l^t = \beta_t m_l^{t-1} + (1-\beta_t) g_l^t$;
    else
      $m_l^t = g_l^t$ (record the gradient directly);
    end if
    $\theta_l^{t+1} = \theta_l^t - \eta_l \mathcal{C}(m_l^t)$ where $\mathcal{C}$ is the column-wise normalization;
  end for
end for
Output: Trained model θ^T.

3. The SCALE Algorithm

The above investigation leads to the following simple optimizer, SCALE, detailed in Algorithm 1. The design of SCALE is motivated by empirical insights from our preceding experiments, which highlighted the importance of stabilizing updates in the last layer and controlling gradient scale via column-wise normalization. Accordingly, SCALE integrates two components: column-wise normalization of the gradients (for all layers) and momentum usage restricted to the last layer. Despite its simplicity, SCALE is highly effective and requires only minimal modifications to standard implementations of Adam—typically just a few additional lines of code. As we will demonstrate, this lightweight design allows SCALE to outperform state-of-the-art baselines such as Adam and achieve performance competitive with Stable-SPAM and Muon, while using substantially less memory.

Connection to existing works. We now give a discussion on the connection of the proposed method to existing works. The proposed algorithm utilizes column-wise normalization, which is also discussed in Pethick et al. (2025). Pethick et al. (2025) uses momentum for all the layers and applies column-wise normalization as an alternative to singular-value normalization only for the last layer. Our method also differs from SWAN (Ma et al., 2025) in the following aspects: first, SWAN utilizes both the row-wise and singular-value normalization while our method only utilizes column-wise normalization, indicating certain redundancy in existing works that apply multiple normalizations; second, SWAN applies Adam for the embedding and LM head layers (the first and last layers) which increases the memory overhead, whereas our approach only introduces first-order momentum for the last layer. We summarize the techniques used in different papers in Table 4.

Convergence of Algorithm 1. The convergence provided in Riabinin et al. (2025, Theorem 2) could be adopted to provide a convergence analysis for Algorithm 1, since Algorithm 1 differs from the general framework of Riabinin et al. (2025, Algorithm 1) only by the fact that we assume different gradient variance σ_i for different layers. Below we provide a convergence result for a generalization of the proposed Algorithm 1:

$$\begin{aligned} &\text{(for each layer } l\text{) } m_l^t = \beta_l m_l^{t-1} + (1-\beta_l) g_l^t \\ &\text{(for each layer } l\text{) } \theta_l^{t+1} = \theta_l^t - \eta_l \mathcal{C}(m_l^t). \end{aligned} \tag{10}$$

We have the following convergence[4]:

Theorem 3.1. *Suppose $\ell(\theta)$ in (1) is lower bounded by ℓ^*, also each layer l is L_l-smooth in matrix $1 \to 2$ norm: $\|\nabla_{\theta_l}\ell(\theta) - \nabla_{\theta_l}\ell(\vartheta)\|_{2\to\infty} \le L_l\|\theta - \vartheta\|_{1\to 2}$, also the stochastic gradient is unbiased $\mathbb{E}_{\xi_t}\nabla_{\theta_l}\ell(\theta^t;\xi_t) = \nabla_{\theta_l}\ell(\theta^t)$ and with bounded variance: $\mathbb{E}_{\xi_t}\| (\nabla_{\theta_l}\ell(\theta^t;\xi_t) - \nabla_{\theta_l}\ell(\theta^t))_{:,j} \|_{2\to\infty}^2 \le \nu_{l,j}^2$ for all $l = 1, ..., L$ and $t = 0, ..., T-1$ and all the columns j. Define $\bar{\sigma}_l := \left(\sum_{j=1}^{n_l} \nu_{l,j}^2\right)^{1/2}$. If $\eta_l = a_l/\sqrt{T}$, we have the following convergence result for (10) (a generalization of Algorithm 1):*

$$\begin{aligned} \frac{1}{T}\sum_{t=1}^{T}\sum_{l=1}^{L} a_l \mathbb{E}\|\nabla_l^t\|_{2\to\infty} &\lesssim \frac{\Delta_1}{T^{1/4}} \\ &+ \sum_{l=1}^{L}\left[\frac{L_l a_l^2}{T^{3/4}} + a_l\left(\frac{\beta_l L_l a_l}{1-\beta_l}\frac{1}{T^{1/2}} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}\frac{1}{T^{1/4}}\right)\right], \end{aligned} \tag{11}$$

where $\nabla_l^t = \nabla_{\theta_l}\ell(\theta^t;\xi_t)$ and $\Delta_1 = \ell(\theta^1) - \ell^*$. Also we omitted constants in $\lesssim$ notation.

[4]Note that to show convergence we need a different set of assumptions (which directly assume smoothness and bounded variance in matrix $1 \to 2$ norm layer-wisely)

Methods	Sign	Col-wise	Row-wise	Singular-val	1st order EMA	2nd order EMA	Memory (7B)
SGD							13.48
Adam	✓				✓	✓	40.43
Muon				✓	✓		26.95
SWAN*			✓	✓	First/Last Layer	First/Last Layer	14.52
APOLLO*					Rank-256	Rank-256	16.14
APOLLO-Mini*					Rank-1	Rank-1	14.53
SCALE		✓			Last Layer		13.74

*: SWAN, APOLLO and APOLLO-Mini apply Adam for the first and last layer.

Table 4. A summary of building components of related methods. "Sign", "Col-wise", "Row-wise" and "Singular-val" correspond to four normalizations in (6), respectively. "1st order EMA" and "2nd order EMA" stand for first and second order EMA/momentum. The last column records the memory (GB) of weights and optimizer states required for applying the corresponding methods for LLaMA 7B training (see Appendix B for the details of the estimation). Note that for the special case of vector parameters (such as LayerNorm weights) all memory-efficient methods employ the Adam optimizer with negligible impact on memory.

4. Experiments

In this section, we test the SCALE algorithm for LLM pretraining. We test on pretraining LLaMA (60M, 130M, 350M, 1B, 7B) models on the C4 (Colossal Clean Crawled Corpus) dataset (Raffel et al., 2020) and report evaluation perplexity as our metric. For all ours experiments, we follow the hyperparameter settings of Zhao et al. (2024); see Appendix C for more details. We include pretraining experiments on different architectures (GPT2-Medium, Qwen2-500M and Gemma-2B) to show the generality of the method in Appendix F.

Baselines. We compare with Adam and its stabilized version, noted as Adam (Stable-SPAM), which performs momentum resets and clips spiked gradients (Huang et al., 2025). Among memory efficient optimizers, we compare with Muon, GaLore, Fira, APOLLO(-Mini) and SWAN (see Section 1.1 for a more detailed introduction of these works). GaLore, Fira and APOLLO(-Mini) compress the Adam states by projecting the gradients to low rank. SWAN is an optimizer that uses both singular-value and row-wise normalization, except for the first and last layers[5].

It is worth noticing that GaLore, Fira, APOLLO(-Mini) and SWAN run Adam for the first and last layers for stable training. For the 60M model the first and last layers contain over 50% of the total network parameters, and around 40% for 130M model. For the 350M this goes down to about 20% and for the 1B to about 10%. Therefore, for smaller models these methods have limited memory savings as compared to Adam, which is used to optimize a significant percentage of the network parameters.

Results for 60M–1B models. We report the results of the evaluation perplexity in Table 5. We can see that, despite its simplicity, SCALE outperforms existing memory efficient optimizers, also (nearly) matches the performance of Adam (Stable-SPAM) and Muon, especially for larger models, with only 35–65% of the memory. We also contrast the performance with the memory consumed in Figure 1, where we can see that the proposed method is at the Pareto frontier for optimal memory usage while maintaining the (near) state-of-the-art performance. This makes it a strong candidate for large-scale pretraining under memory constraints[6].

Model Size	60M	130M	350M	1B
Tokens	1.4B	2.6B	7.8B	20B
Adam†	30.05 (0.35G)	23.13 (0.81G)	18.77 (2.21G)	15.79 (8.04G)
Adam (Stable-SPAM)†	28.77 (0.35G)	22.20 (0.81G)	16.80 (2.21G)	13.30 (8.04G)
Muon	**28.86** (0.23G)	**22.20** (0.54G)	16.70 (1.47G)	13.67 (5.36G)
GaLore†	34.58 (0.28G)	25.31 (0.61G)	19.37 (1.59G)	15.05 (4.76G)
Fira†	30.34 (0.28G)	22.96 (0.61G)	16.82 (1.59G)	14.36 (4.76G)
SWAN*	30.00 (0.25G)	22.83 (0.46G)	17.14 (1.00G)	-
APOLLO	30.94 (0.28G)	22.93 (0.61G)	16.75 (1.59G)	14.28 (4.76G)
APOLLO-Mini	31.85 (0.25G)	23.63 (0.46G)	17.11 (1.00G)	**13.48** (3.20G)
SCALE (ours)	30.81 (**0.15G**)	22.57 (**0.32G**)	**16.32** (**0.80G**)	**13.49** (**2.81G**)

Table 5. Experiment results for pretraining LLaMA models on C4 dataset. The result marked * is from Ma et al. (2025). The results marked † for model sizes 60M-350M are from Glentis et al. (2025). All models are trained up-to the Chinchilla optimal number of tokens (Hoffmann et al., 2022). For Fira and APOLLO 1B runs we encountered training divergence with their default learning rates, results reported are with reduced learning rate. Among the memory-efficient optimizers we highlight the best-performing for each model size in terms of perplexity and memory. Note that SWAN does not provide a 1B model result with Chinchilla optimal tokens, but only for 13B tokens. In that setting we also achieve a superior perplexity of 14.25 (2.81G memory), compared to 14.42 (3.20G memory) of SWAN.

[5]We copy SWAN's result from (Ma et al., 2025) since we cannot replicate them due to code unavailability.

[6]Notice that we construct our optimizer step by step via a minimalist design, and we already conduct the ablations throughout our journey (see Table 2 and 3, also Figure 4). We do not conduct further ablation studies in this section, but we include experiments on throughput analysis, additional architectures, overtraining experiment, finetuning experiments and learning rate sensitivity in the Appendix sections D, F, H, I and K, respectively.

Steps		40K	80K	120K	150K (final)
Tokens		5.2B	10.5B	15.7B	19.7B
APOLLO†	(16.14G)	17.55	14.39	13.23	13.02
APOLLO-Mini†	(14.53G)	18.03	14.60	13.32	13.09
Muon	(26.95G)	16.43	13.95	12.85	12.72
SCALE (ours)	(**13.74G**)	17.99	14.57	12.86	**12.59**

Table 6. Results for pretraining 7B LLaMA model on C4 dataset. †From Zhu et al. (2025).

Results for 7B LLaMA model. Due to limited computational resources, we run a single experiment for SCALE and Muon on 8×NVIDIA H200 141G GPUs. To compare with the reported results from APOLLO(-Mini) (Zhu et al., 2025), we train for a total of 19.7B tokens, corresponding to 150K steps. We report the final evaluation perplexity in Table 6 as well as perplexity at intermediate training steps. From the table, we conclude that SCALE outperforms Muon and APOLLO(-Mini) in terms of both final evaluation perplexity and memory. In addition, we verified the stability of SCALE during an extended 100B token pretraining run of the 7B model; see Appendix G for details.

5. Conclusion

In this paper, we design a memory-efficient optimizer using a minimalist approach. The proposed algorithm utilizes the building blocks that lead to the success of Adam but further refines them to make it more memory efficient. We motivate each of our construction steps by theoretical or experimental evidences. The resulting algorithm, SCALE, achieves superior performance to existing memory-efficient optimizers for LLM pretraining and matches Adam while only requiring 35-45% of memory. This makes the proposed algorithm a strong candidate for large-scale pretraining under memory constraints, as well as a minimalist baseline for benchmarking more sophisticated optimizers.

One limitation of our current study is that, due to computational constraints, we evaluate our optimizer on models up to 7B parameters and up to 100B training tokens. While such model sizes and token budgets are on par with comparisons in related literature in memory-efficient pretraining, an evaluation to even larger models and more tokens will be helpful to further show the benefits of the proposed optimizer.

Acknowledgements

M. Hong and A. Glentis are partly supported by NSF Awards ECCS-2426064 and EPCN-2311007.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. In particular, by making LLM pretraining more memory- and compute-efficient, the barrier to training larger models can be lowered. This has beneficial consequences including democratizing LLM development, but also includes potential risks from the wider access to powerful LLMs.

References

Anil, R., Gupta, V., Koren, T., and Singer, Y. Memory efficient adaptive optimization. *Advances in Neural Information Processing Systems*, 32, 2019.

Bernstein, J. and Newhouse, L. Modular duality in deep learning. *arXiv preprint arXiv:2410.21265*, 2024a.

Bernstein, J. and Newhouse, L. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024b.

Bernstein, J. and Newhouse, L. Modular duality in deep learning. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=hErdffTsLu.

Chen, X., Feng, K., Li, C., Lai, X., Yue, X., Yuan, Y., and Wang, G. Fira: Can we achieve full-rank training of llms under low-rank constraint? *arXiv preprint arXiv:2410.01623*, 2024.

Cutkosky, A. and Mehta, H. Momentum improves normalized sgd. In *International conference on machine learning*, pp. 2260–2268. PMLR, 2020.

Glentis, A., Li, J., Shang, Q., Han, A., Tsaknakis, I., Wei, Q., and Hong, M. Scalable parameter and memory efficient pretraining for llm: Recent algorithmic advances and benchmarking. *arXiv preprint arXiv:2505.22922*, 2025.

Han, A., Li, J., Huang, W., Hong, M., Takeda, A., Jawanpuria, P., and Mishra, B. SLTrain: a sparse plus low rank approach for parameter and memory efficient pretraining. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=MXze4H7opg.

Hinton, G., Srivastava, N., and Swersky, K. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Cited on*, 14(8):2, 2012.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 30016–30030, 2022.

Huang, T., Hu, H., Zhang, Z., Jin, G., Li, X., Shen, L., Chen, T., Liu, L., Wen, Q., Wang, Z., et al. Stable-spam: How to train in 4-bit more stably than 16-bit adam. *arXiv preprint arXiv:2502.17055*, 2025.

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

Kalra, D. S., Kirchenbauer, J., Barkeshli, M., and Goldstein, T. When can you get away with low memory adam? *arXiv preprint arXiv:2503.01843*, 2025.

Kingma, D. P. and Ba, J. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.

Kudo, T. and Richardson, J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226*, 2018.

Kunstner, F., Chen, J., Lavington, J. W., and Schmidt, M. Noise is not the main factor behind the gap between sgd and adam on transformers, but sign descent might be. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=a65YK0cqH8g.

Kunstner, F., Milligan, A., Yadav, R., Schmidt, M., and Bietti, A. Heavy-tailed class imbalance and why adam outperforms gradient descent on language models. *Advances in Neural Information Processing Systems*, 37: 30106–30148, 2024.

Levy, K. Y. The power of normalization: Faster evasion of saddle points. *arXiv preprint arXiv:1611.04831*, 2016.

Liao, X., Li, S., Xu, Y., Li, Z., Liu, Y., and He, Y. Galore +: Boosting low-rank adaptation for llms with cross-head projection. *arXiv preprint arXiv:2412.19820*, 2024.

Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Liu, Y., Gao, Y., and Yin, W. An improved analysis of stochastic gradient descent with momentum. *Advances in Neural Information Processing Systems*, 33:18261–18271, 2020.

Luo, Q., Yu, H., and Li, X. Badam: A memory efficient full parameter training method for large language models. *arXiv preprint arXiv:2404.02827*, 2024.

Luo, Y., Ren, X., Zheng, Z., Jiang, Z., Jiang, X., and You, Y. Came: Confidence-guided adaptive memory efficient optimization. *arXiv preprint arXiv:2307.02047*, 2023.

Ma, C., Gong, W., Scetbon, M., and Meeds, E. SWAN: SGD with normalization and whitening enables stateless LLM training. In *Forty-second International Conference on Machine Learning*, 2025.

Muhamed, A., Li, O., Woodruff, D., Diab, M., and Smith, V. GRASS: Compute efficient low-memory llm training with structured sparse gradients. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 14978–15003, 2024.

Murray, R., Swenson, B., and Kar, S. Revisiting normalized gradient descent: Fast evasion of saddle points. *IEEE Transactions on Automatic Control*, 64(11):4818–4824, 2019.

Pan, R., Liu, X., Diao, S., Pi, R., Zhang, J., Han, C., and Zhang, T. Lisa: layerwise importance sampling for memory-efficient large language model fine-tuning. *Advances in Neural Information Processing Systems*, 37: 57018–57049, 2024.

Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., and Cevher, V. Training deep learning models with norm-constrained lmos. *arXiv preprint arXiv:2502.07529*, 2025.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140):1–67, 2020.

Ramesh, A. V., Ganapathiraman, V., Laradji, I. H., and Schmidt, M. Blockllm: Memory-efficient adaptation of llms by selecting and optimizing the right coordinate blocks. *arXiv preprint arXiv:2406.17296*, 2024.

Reddi, S. J., Kale, S., and Kumar, S. On the convergence of adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2019.

Riabinin, A., Shulgin, E., Gruntkowska, K., and Richtárik, P. Gluon: Making muon & scion great again!(bridging theory and practice of lmo-based optimizers for llms). *arXiv preprint arXiv:2505.13416*, 2025.

Scetbon, M., Ma, C., Gong, W., and Meeds, E. Gradient multi-normalization for stateless and scalable llm training. *arXiv preprint arXiv:2502.06742*, 2025.

Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.

Sun, T., Liu, X., and Yuan, K. Revisiting gradient normalization and clipping for nonconvex sgd under heavy-tailed noise: Necessity, sufficiency, and acceleration. *Journal of Machine Learning Research*, 26:1–42, 2025.

Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, pp. 353–355, 2018.

Xu, M., Xiang, L., Cai, X., and Wen, H. No more adam: Learning rate scaling at initialization is all you need. *arXiv preprint arXiv:2412.11768*, 2024.

Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Tang, J., Wang, J., Yang, J., Tu, J., Zhang, J., Ma, J., Yang, J., Xu, J., Zhou, J., Bai, J., He, J., Lin, J., Dang, K., Lu, K., Chen, K., Yang, K., Li, M., Xue, M., Ni, N., Zhang, P., Wang, P., Peng, R., Men, R., Gao, R., Lin, R., Wang, S., Bai, S., Tan, S., Zhu, T., Li, T., Liu, T., Ge, W., Deng, X., Zhou, X., Ren, X., Zhang, X., Wei, X., Ren, X., Liu, X., Fan, Y., Yao, Y., Zhang, Y., Wan, Y., Chu, Y., Liu, Y., Cui, Z., Zhang, Z., Guo, Z., and Fan, Z. Qwen2 technical report, 2024. URL https://arxiv.org/abs/2407.10671.

Zhang, J., He, T., Sra, S., and Jadbabaie, A. Why gradient clipping accelerates training: A theoretical justification for adaptivity. In *International Conference on Learning Representations*, 2020a. URL https://openreview.net/forum?id=BJgnXpVYwS.

Zhang, J., Karimireddy, S. P., Veit, A., Kim, S., Reddi, S., Kumar, S., and Sra, S. Why are adaptive methods good for attention models? *Advances in Neural Information Processing Systems*, 33:15383–15393, 2020b.

Zhang, Y., Chen, C., and Luo, Z.-Q. Does adam converge and when? In *ICLR Blog Track*, 2022a. URL https://iclr-blog-track.github.io/2022/03/25/does-adam/. https://iclr-blog-track.github.io/2022/03/25/does-adam/.

Zhang, Y., Chen, C., Shi, N., Sun, R., and Luo, Z.-Q. Adam can converge without any modification on update rules. *Advances in neural information processing systems*, 35: 28386–28399, 2022b.

Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024.

Zhao, J., Zhang, Z., Chen, B., Wang, Z., Anandkumar, A., and Tian, Y. Galore: Memory-efficient LLM training by gradient low-rank projection. In *Forty-first International Conference on Machine Learning*, 2024.

Zhao, R., Morwani, D., Brandfonbrener, D., Vyas, N., and Kakade, S. M. Deconstructing what makes a good optimizer for autoregressive language models. In *The Thirteenth International Conference on Learning Representations*, 2025.

Zhu, H., Zhang, Z., Cong, W., Liu, X., Park, S., Chandra, V., Long, B., Pan, D. Z., Wang, Z., and Lee, J. APOLLO: SGD-like memory, adamw-level performance. In *Eighth Conference on Machine Learning and Systems*, 2025. URL https://openreview.net/forum?id=mJrPkdcZDj.

A. LLM Acknowledgment

We acknowledge the use of LLMs for grammar checking and sentence polishing.

B. Details of memory estimation for 1B and 7B models

Here we compute the memory estimate for both 1B and 7B LLaMA models. We only compute the major parameters, including embedding layers, attention and MLP layers. We follow prior works (Zhao et al., 2024; Han et al., 2024) in estimating the memory using `bfloat16` format, where each floating point number occupies 2 bytes.

7B model: Pre-last layers include 6.607B parameters and last layer includes 0.131B parameters, which in total leads to 6.738B parameters.

- **SGD**: Only the parameter states are stored, which amount to 13.476G memory.
- **Adam**: Apart from the parameter states, Adam/AdamW store first and second order momentum, which costs 26.952G. In total, Adam/AdamW requires 40.428G memory.
- **APOLLO**: Apart from the parameter states, APOLLO stores first-order and second-order momentum in the low-rank subspace of 256, which in total costs 16.144G.
- **APOLLO-Mini**: Apart from the parameter states, APOLLO-Mini sets rank to be 1, which leads to a total memory of 14.531G.
- **Muon**: Apart from the parameter states, Muon stores first-order momentum, which costs 13.476G. In total, Muon requires 26.952G memory.
- **SWAN**: Apart from the parameter states, SWAN additionally stores first-order and second-order momentum of the first and last layer, which costs 1.048G. In total, SWAN requires 14.524G.
- **`SCALE`** (Our method): Apart from parameter states, `SCALE` additionally stores first-order momentum of last-layer weight, which costs 0.262G. In total, `SCALE` requires 13.738G memory.

1B model: Pre-last layers include 1.273B parameters and last layer includes 0.066B parameters, which in total leads to 1.339B parameters.

- **SGD**: Only the parameter states are stored, which amount to 2.678G memory.
- **Adam**: Apart from the parameter states, Adam/AdamW store first and second order momentum, which costs 5.356G. In total, Adam/AdamW requires 8.034G memory.
- **Muon**: Apart from the parameter states, Muon stores first-order momentum, which costs 2.678G. In total, Muon requires 5.356G memory.
- **SWAN**: Apart from the parameter states, SWAN additionally stores first-order and second-order momentum of the first and last layer, which costs 0.524G. In total, SWAN requires 3.202G.
- **`SCALE`** (Our method): Apart from parameter states, `SCALE` additionally stores first-order momentum of last-layer weight, which costs 0.131G. In total, `SCALE` requires 2.809G memory.

C. Details of the experiments

For all LLaMA experiments, we follow (Zhao et al., 2024) and set the sequence length to 256 and the batch size to 512, train using BF16 format and report evaluation perplexity as our metric. We also use a cosine learning rate with linear warm-up for the first 10% of the iterations. For low-rank optimizers (GaLore, Fira and APOLLO) we follow their suggested hyperparameters (including the rank) but tune the learning rates. For Muon we follow the implementation from (Liu et al., 2025).

We performed wandb sweeps for all methods we tested up to models of size 350M, searching learning rates within {0.00005, 0.0001, 0.0003, 0.0005, 0.001, 0.003, 0.005, 0.01}. For the 1B model, due to resource constraints we manually tune the learning rates using as starting point the optimal learning rate from the 350M sweep. For SCALE, we use learning rate 1e-3 for models sizes 60M, 130M and 350M, 2e-4 for the 1B and 1e-4 for the 7B model. In addition, our reported result for the 1B model uses the same learning rate scaling technique used by Muon (Liu et al., 2025). Also, we set the last layer's momentum parameter $\beta = 0.9$, being a common choice for first-order momentum. In addition, for all vector parameters we employ the Adam optimizer, following (Jordan et al., 2024; Liao et al., 2024). This does not influence the memory usage because the vector parameters are orders of magnitude smaller in size compared to the matrix parameters.

D. Training Throughput Comparison of Different Methods

We conduct a throughput (tokens/sec) comparison of the different optimizers for training LLaMA 1B on 4xH100 GPUs using the same settings as specified in C. Table 7 shows that our method is about 18.5% faster than singular-value (NS) based normalization methods (Muon/SWAN) and about 8% faster than GaLore/Fira which project the Adam states to low-rank using SVD. Moreover, it achieves similar throughput to Adam/Stable-SPAM and APOLLO(-Mini).

Method	Throughput (tokens/sec)
Adam	45019
Adam (Stable-SPAM)	44960
Singular-value-based (Muon/SWAN*)	37748
GaLore	41267
Fira	41285
APOLLO	44193
APOLLO-Mini	44567
SCALE	44728

Table 7. Throughput comparison of different methods for training LLaMA 1B on 4xH100 GPUs. * : Due to code unavailability we cannot directly test the throughput of SWAN, instead we report the throughput using the Newton-Schulz (NS) approximation of Singular-value normalization (see (Jordan et al., 2024) for details).

E. Ablation on Adding Momentum to the First Layer

We ablate using momentum both to the first and the last layer (SGD column-wise mmt-(first+last)) and include the results in the Table 8 bellow:

Method / LLaMA Model	60M	130M	350M
SGD column-wise (no momentum)	39.89 (0.12G)	28.85 (0.27G)	20.38 (0.74G)
SGD column-wise mmt-last (SCALE)	30.81 (0.15G)	22.57 (0.32G)	16.32 (0.80G)
SGD column-wise mmt-(first+last)	30.35 (0.18G)	22.26 (0.37G)	16.14 (0.87G)

Table 8. Experiments exploring the benefit of adding momentum to the embedding layer. The memory is included in parenthesis. The additional momentum for the first layer does not result to significant gains in model performance, which further validates our optimizer design choices.

F. Pretraining Results on Additional LLM Architectures

For our main experiments we choose the LLaMA family of models as it is commonly used to benchmark memory efficient optimizers for LLM pretraining (such as in GaLore (Zhao et al., 2024), Fira (Chen et al., 2024), SWAN (Ma et al., 2025) and APOLLO (Zhu et al., 2025)), which is the scope of our paper. However, to provide further evidence of the generality of our algorithm among LLM architectures, we conducted additional pretraining experiments with Qwen2-500M (Yang

et al., 2024), GPT2-Medium (355M) (Radford et al., 2019) and Gemma-2B (Team et al., 2024). We followed the same experimental settings as in our LLaMA experiments, described in C. We provide the results bellow:

Model	Qwen2-500M	GPT2-M (355M)
Tokens	10B	7.8B
Adam	17.61 (2.96G)	20.73 (2.13G)
Adam (Stable-SPAM)	15.91 (2.96G)	18.90 (2.13G)
Muon	16.03 (1.98G)	19.61 (1.42G)
GaLore	18.22 (1.94G)	23.66 (1.22G)
Fira	15.94 (1.94G)	19.41 (1.22G)
APOLLO	16.04 (1.94G)	19.30 (1.22G)
APOLLO-Mini	16.17 (1.53G)	19.99 (0.92G)
SCALE	**15.57** (**1.26G**)	**19.00** (**0.81G**)

Table 9. Experiment results from pretraining Qwen and GPT2 models on the C4 dataset.

From Table 9 we observe that our method still achieves Adam-like performance while using significantly less memory and continues to outperform SOTA memory-efficient methods. We believe that our simple, yet systematic, optimizer design makes our method more robust and less likely to overfit to specific LLM architectures.

Model	Gemma-2B
Tokens	32B
APOLLO	12.05 (9.09G)
SCALE	**11.96** (**6.06G**)

Table 10. Experiment results from pretraining Gemma-2B on the C4 dataset. Due to resource and time constrains we limit our comparison to APOLLO, being the strongest baseline.

From table 10 we can see that our method achieves a lower perplexity with an even smaller memory footprint than APOLLO, which further showcases its SOTA performance as a memory-efficient pretraining algorithm.

G. Results from Training LLaMA 7B up to 100B Tokens

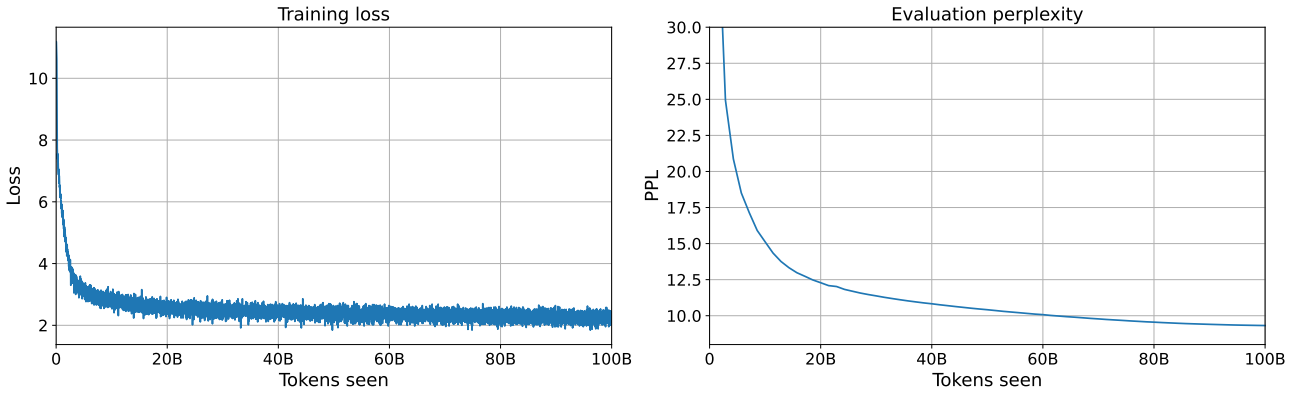


Figure 5. Training 7B LLaMA on C4 up-to 100B tokens using the SCALE optimizer. The final evaluation perplexity achieved is 9.32.

To further validate the stability of SCALE in (near) industry-level training scope we pre-trained the LLaMA 7B model on C4 up-to 100B tokens (near the Chinchilla optimal training budget). We use the same settings as in our 7B experiment in Table 6, with the differences being scaling to two 8xH200 nodes with distributed data parallel, increased batch size of 1024 to fully utilize the GPU memory and learning rate increased by $\sqrt{2}$ to $\sqrt{2} \times 1e-4$ to account for the doubled batch size.

Using the SCALE optimizer we were able to achieve a single-digit final evaluation perplexity of 9.32, while maintaining a stable loss trajectory throughout training (fully absent of loss spikes), as shown in Figure 5. This result validates our optimizer's stability in large model and high budget runs.

H. Results on Overtraining Regime

Chinchilla ratio		1×	2×	4×
Adam	(2.21G)	18.77	17.60	17.21
Adam (Stable-SPAM)	(2.21G)	16.80	15.85	15.11
Muon	(1.47G)	16.70	15.81	15.18
GaLore	(1.59G)	19.37	18.40	17.81
Fira	(1.59G)	16.82	15.82	15.31
APOLLO	(1.59G)	16.75	15.76	15.06
APOLLO-Mini	(1.00G)	17.11	16.02	15.21
SCALE	**(0.80G)**	**16.32**	**15.33**	**14.77**

Table 11. Results from training the 350M LLaMA model on C4 using different token budgets. Chinchilla ratio 1× corresponds to (roughly) our default token budget for the given model, i.e., 7.8B tokens (following (Zhao et al., 2024)), 2× to 15.6B tokens and 4× to 31.2B tokens. Our method maintains its SOTA memory-efficient pretraining performance among the different training budgets.

I. Fine-tuning Results

First, we want to emphasize that our optimizer design is focused on memory-efficient LLM pretraining. As pretraining is by far the most computationally expensive part of the LLM training process (for example pretraining LLaMA2-70B took about 1.72 million A100 GPU hours (Touvron et al., 2023)), it is commonly believed that an efficient SOTA pretraining optimizer is of great importance.

Instead, for fine-tuning, existing Parameter-efficient fine-tuning (PEFT) methods already reduce the memory and compute requirements substantially while giving performance competitive to full fine-tuning. Therefore we don't expect our optimizer to replace such approaches. However, to give further evidence of its generality, we provide some preliminary results on GLUE benchmarks with a pretrained RoBERTa-base (Liu et al., 2019) which we fine-tuned with our method and Adam (full fine-tuning), following the setup of (Zhao et al., 2024). We conduct a learning rate search for both methods and report the best epoch results (higher the better).

Method / Benchmark		RTE	CoLA	MRPC	STS-B	SST-2	QNLI	**Avg**
Adam	(0.75G)	79.53	63.37	93.01	91.26	94.26	92.67	85.68
SCALE	(0.33G)	80.46	63.02	92.53	91.23	93.58	92.23	85.51

Table 12. Results from fine-tuning RoBERTa-base on different GLUE benchmarks (Wang et al., 2018). Memory consumption is indicated in the parenthesis.

We observe that our method, despite being designed for pretraining, can give results comparable to Adam for fine-tuning tasks while using over 2 times less memory (included in parenthesis). Again, we want to point out that our optimizer is aimed at memory-efficient pretraining where its advantages are most relevant. As a future work, it might be worth investigating combining SCALE with PEFT methods, although this is out of the scope of this paper.

J. More Variance Analysis Results

In addition to our variance analysis in Figure 4, we ablate the impact of model size, architecture and tokenizer to the per-layer gradient variance behavior. As shown in Figures 6 and 7, The pattern remain similar to Figure 4, i.e., the last layer exhibits significantly more gradient variance than all other layers, and adding momentum to it helps reduce its variance.

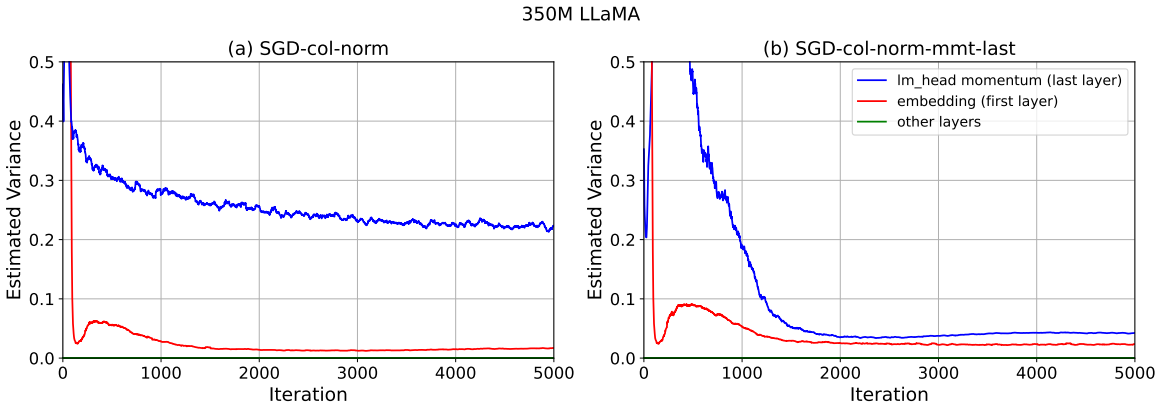


Figure 6. Variance analysis experiment for a 350M LLaMA model on C4.

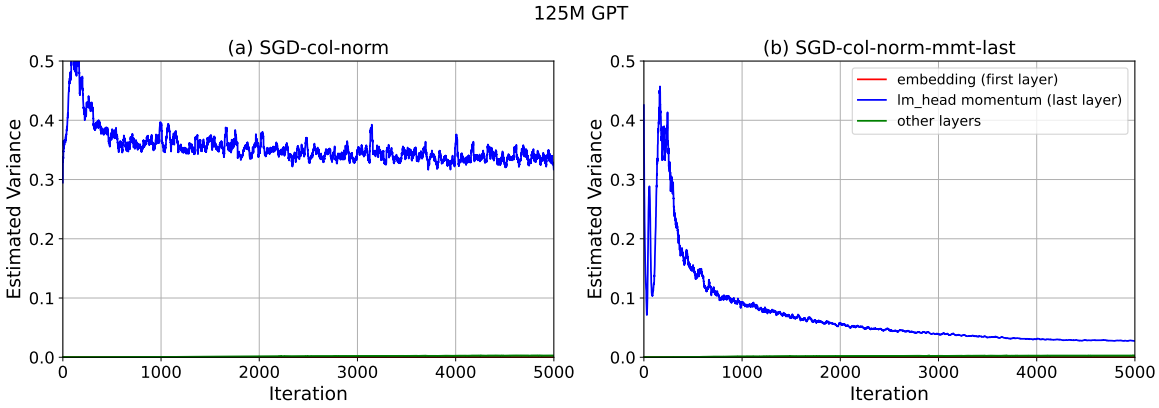


Figure 7. Variance analysis experiment for a GPT-125M model using the GPT-2 tokenizer on C4.

K. Learning Rate Sensitivity Analysis

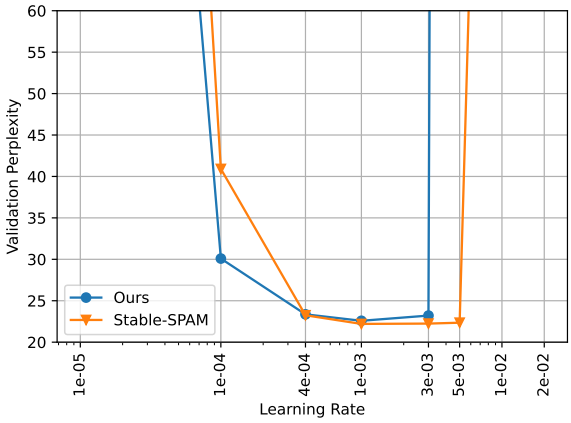


Figure 8. Learning rate sensitivity analysis, comparing Stable-SPAM (a stabilized version of Adam) and our method. Results from the 130M LLaMA model.

In Figure 8 we test the performance of our algorithm, SCALE, with different learning rates and compare it with that of Adam (Stable-SPAM version) (Huang et al., 2025). We observe that both algorithms behave similarly with a reasonable range of learning rates.

L. Curves of Training Iteration versus Perplexity

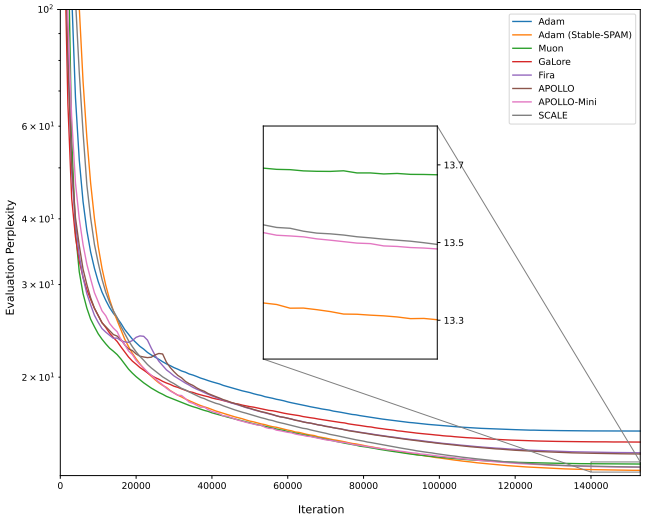
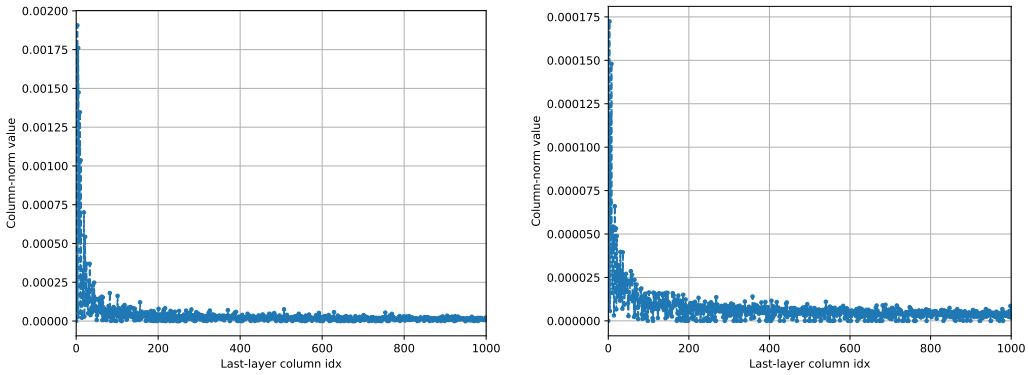


Figure 9. The evaluation perplexity curves of different methods on LLaMA 1B pretraining. Note that Muon is converging the fastest at the beginning stage, while SCALE, Adam (Stable-SPAM) and APOLLO-Mini catch up in the final stage of training.

M. Discussion on why Column-wise matters for the Last Layer



(a) Iteration 1,000. *(b)* Iteration 10,000.

Figure 10. The column-norm values of the last layer's gradient matrix at iterations 1,000 and 10,000 during the training of LLaMA 130M. The last layer of LLaMA 130M has gradient dimensions $d_{\text{model}} \times |V| = 768 \times 32,000$, which means that there are $|V| = 32,000$ columns (and therefore $|V|$ corresponding column-norms). For clarity we limit the x axis to the first 1000 columns. We also note that due to the design of the SentencePiece tokenizer (Kudo & Richardson, 2018) used, lower token ids (and therefore lower last-layer column-ids) generally correspond to more frequent tokens (such as "_the", "_and", "_to", etc., as we observed). Therefore, from the above figures it can be seen that more frequent tokens have much larger column-norms, potentially leading to imbalanced learning. We hypothesize that the effectiveness of column-wise is related to mitigating this phenomenon by normalizing all the column norms to the same level and thus enabling more balanced training dynamics.

In this section we further investigate the importance of column-wise normalization for the last layer, and also ablate several "mixed" normalization schemes, where more than one type of normalization is used depending on the layer.

Method	Perplexity
1. SCALE (all column, mmt-last)	**22.57**
2. column-last, row-rest, mmt-last	23.27
3. row-first, column-rest, mmt-last	22.94
4. norm along larger dim, mmt-last	23.52
5. row-last, column-rest, mmt-last	28.83

Table 13. Ablation results from training the 130M LLaMA model on C4 using "mixed" normalization schemes comapred to SCALE.

All results of Table 13 are from pretraining the LLaMA 130M model using momentum only for the last layer (mmt-last, as in SCALE). Methods 1-4 all do column-wise normalization for the last layer; only method 5 (row-last, column-rest, mmt-last) does row-wise for the last layer (row-last). Note that "column-rest" or "row-rest" means we do column- or row-wise normalizations for all other layers we don't explicitly state.

We can clearly observe from the above results that row-last significantly degrades the performance. In addition, we can see that excluding the last layer, the rest of the layers are less influenced by the normalization direction. However, a uniform normalization approach as in SCALE (all column-wise) still outperforms other "mixed" normalization approaches.

To explain the above results, our intuition suggests that the last layer in LLMs is somehow "special". Indeed, it is the linear projection layer (often called Language modeling head) that maps the last hidden state (of dimension d_{model}) to the vocabulary logits (of dimension $|V|$ being the vocabulary size). Its gradient matrix shape is $d_{model} \times |V|$, i.e., it has $|V|$ columns, same as its weight matrix shape. Therefore, each gradient column of the last layer has a "physical meaning", i.e., it corresponds to one of the $|V|$ vocabulary tokens.

We have observed that for certain (few) tokens, their corresponding last-layer column gradients have much larger norms compared to the rest. We have also observed that such columns correspond to more frequent tokens (see Figure 10 details). We hypothesize that such last-layer column-norm differences can potentially lead to uneven learning (i.e., more rare tokens receive smaller gradients and therefore are not "learned" by the model) or even training divergence (due to the large gradients corresponding to frequent tokens). In connection to this, (Kunstner et al., 2024), aiming at explaining the large Adam-SGD gap in LLM training, observed that SGD is able to make much less progress on learning low-frequency classes compared to Adam. Based on those insights, we conclude that by normalizing the columns of the last layer our method obtains more stable training dynamics and a more "even" token learning behavior, even without using full per-parameter adaptivity as in Adam.

Although still at a preliminary stage, we believe that our above insights for the importance of column-wise normalization for the last layer can provide a deeper understanding of the normalization component and (at least to some degree) explain why SCALE is able to perform comparable to Adam for LLM pretraining.

N. Proofs for Section 2.2

In this section, we conduct the proof for Theorem 2.1. The proof follows Liu et al. (2020). First, we have the following lemmas, which are variations of Liu et al. (2020, Lemma 1 and 2).

Lemma N.1. *Suppose that the assumptions in Theorem 2.1 hold. For SGD-M (7), we have*

$$\mathbb{E}\left[\left\|m_l^t - (1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right] \le \frac{1-\beta_l}{1+\beta_l}\left(1-\beta_l^{2t}\right)\sigma_l^2. \tag{12}$$

Proof. It is straightforward to see $m_l^t = (1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}g_l^i$, where $g_l^i := \nabla_{\theta_l} f(\theta^i;\xi_i)$.

We have

$$\mathbb{E}\left[\left\|m_l^t - (1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right] = (1-\beta_l)^2\mathbb{E}\left\|\sum_{i=1}^{t}\beta_l^{t-i}(g_l^i - \nabla_{\theta_l}\ell(\theta^i))\right\|^2.$$

Therefore

$$\begin{aligned}
&\mathbb{E}\left[\left\|m_l^t - (1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right] \\
=&(1-\beta_l)^2\mathbb{E}_{\xi_1}\mathbb{E}_{\xi_2}\cdots\mathbb{E}_{\xi_t}\left\|\sum_{i=1}^{t}\beta_l^{t-i}(g_l^i - \nabla_{\theta_l}\ell(\theta^i))\right\|^2 \\
=&(1-\beta_l)^2\mathbb{E}_{\xi_1}\mathbb{E}_{\xi_2}\cdots\mathbb{E}_{\xi_t}\left[\sum_{i=1}^{t}\sum_{j=1}^{t}\langle\beta_l^{t-i}(g_l^i - \nabla_{\theta_l}\ell(\theta^i)), \beta_l^{t-j}(g_l^j - \nabla_{\theta_l}\ell(\theta^j))\rangle\right].
\end{aligned}$$

Due to unbiasedness of stochastic gradients, the cross terms cancel, therefore we have

$$\begin{aligned}&\mathbb{E}\left[\left\|m_l^t-(1-\beta_l)\sum_{i=1}^t\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right]\\=&(1-\beta_l)^2\sum_{i=1}^t\beta_l^{2(t-i)}\mathbb{E}_{\xi_1}\mathbb{E}_{\xi_2}\cdots\mathbb{E}_{\xi_t}\left\|g_l^i-\nabla_{\theta_l}\ell(\theta^i)\right\|^2\le\frac{1-\beta_l}{1+\beta_l}(1-\beta_l^{2t})\sigma_l^2,\end{aligned}$$

for all layers $l=1,...,L$. □

Lemma N.2. *Suppose that the assumptions in Theorem 2.1 hold. For SGD-M (7), we have*

$$\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^t\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)-\nabla_{\theta_l}\ell(\theta^t)\right\|^2\right]\le\sum_{i=1}^{t-1}a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right],\tag{13}$$

for all layers $l=1,...,L$, where

$$a_{l,t,i}=\frac{\gamma^2\beta_l^{t-i}}{1-\beta_l^t}\left(t-i+\frac{\beta_l}{1-\beta_l}\right).\tag{14}$$

Proof. Since

$$\begin{aligned}&\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^t\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)-\nabla_{\theta_l}\ell(\theta^t)\right\|^2\right]\\&=\left(\frac{1-\beta_l}{1-\beta_l^t}\right)^2\sum_{i,j=1}^t\mathbb{E}\left[\left\langle\beta_l^{t-i}\left(\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^i)\right),\beta_l^{t-j}\left(\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^j)\right)\right\rangle\right]\\&\le\left(\frac{1-\beta_l}{1-\beta_l^t}\right)^2\sum_{i,j=1}^t\beta_l^{2t-i-j}\left(\frac{1}{2}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right]+\frac{1}{2}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^j)\right\|^2\right]\right)\\&=\left(\frac{1-\beta_l}{1-\beta_l^t}\right)^2\sum_{i=1}^t\left(\sum_{j=1}^t\beta_l^{2t-i-j}\right)\frac{1}{2}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^j)\right\|^2\right]\\&\quad+\left(\frac{1-\beta_l}{1-\beta_l^t}\right)^2\sum_{j=1}^t\left(\sum_{i=1}^t\beta_l^{2t-i-j}\right)\frac{1}{2}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right]\\&=\left(\frac{1-\beta_l}{1-\beta_l^t}\right)^2\sum_{i=1}^t\frac{\beta_l^{t-i}\left(1-\beta_l^t\right)}{1-\beta_l}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right]\\&=\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^t\beta_l^{t-i}\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^t)-\nabla_{\theta_l}\ell(\theta^i)\right\|^2\right]\\&\le\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^t\beta_l^{t-i}(t-i)\sum_{j=i}^t\mathbb{E}\left[\left\|\nabla_{\theta_l}\ell(\theta^{j+1})-\nabla_{\theta_l}\ell(\theta^j)\right\|^2\right]\end{aligned}$$

where we use Cauchy-Schwarz inequality for the first inequality and the last is by AM-GM inequality. Now applying the

Lipschitz smoothness assumption, we have

$$\begin{aligned}
&\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta_l^{t-i}\nabla_{\theta_l}\ell(\theta^i)-\nabla_{\theta_l}\ell(\theta^t)\right\|^2\right]\\
&\le\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta_l^{t-i}(t-i)\sum_{j=i}^{t}\gamma^2\mathbb{E}\left[\left\|\theta^{j+1}-\theta^j\right\|^2\right]\\
&\le\frac{1-\beta_l}{1-\beta_l^t}\sum_{j=1}^{t-1}\left(\sum_{i=1}^{j}\beta_l^{t-i}(t-i)\right)\gamma^2\mathbb{E}\left[\left\|\theta^{j+1}-\theta^j\right\|^2\right].
\end{aligned}$$

Now define

$$a'_{l,t,i}=\frac{1-\beta_l}{1-\beta_l^t}\gamma^2\sum_{i=1}^{j}\beta_l^{t-i}(t-i)=\frac{\gamma^2\beta_l^t}{1-\beta_l^t}\left(-(t-1)-\frac{1}{1-\beta_l}\right)+\frac{\gamma^2\beta_l^{t-j}}{1-\beta_l^t}\left(t-j+\frac{\beta_l}{1-\beta_l}\right)\le a_{l,t,i},$$

then we get (13). □

Lemma N.3. *For SGD-M (7), define the auxiliary sequence by*

$$z_l^t:=\begin{cases}\theta_l^t & t=1\\ \frac{1}{1-\beta_l}\theta_l^t-\frac{\beta_l}{1-\beta_l}\theta_l^{t-1} & t\ge 2\end{cases}\tag{15}$$

and denote $z^t=[z_1^t,...,z_L^t]$ the entire auxiliary variable at iteration t. Then we have

$$z_l^{t+1}-z_l^t=-\eta_l g_l^t$$

and

$$z_l^t-\theta_l^t=-\frac{\beta_l}{1-\beta_l}\eta_l m_l^{t-1}.$$

Proof. For $t=1$, we have (since $m^0=0$)

$$z_l^2-z_1^1=\frac{1}{1-\beta_l}\theta_l^2-\frac{\beta_l}{1-\beta_l}\theta_l^1-\theta_l^1=\frac{1}{1-\beta_l}(\theta_l^2-\theta_l^1)=-\eta_l g_l^1.$$

For $t\ge 2$, we have

$$\begin{aligned}
z_l^{t+1}-z_1^t=&\frac{1}{1-\beta_l}(\theta_l^{t+1}-\theta_l^t)-\frac{\beta_l}{1-\beta_l}(\theta_l^t-\theta_l^{t-1})\\
=&\frac{1}{1-\beta_l}(-\eta_l m_l^t)-\frac{\beta_l}{1-\beta_l}(-\eta_l m_l^{t-1})\\
=&\frac{1}{1-\beta_l}(-\eta_l m_l^t+-\eta_l\beta_l m_l^{t-1})\\
=&-\eta_l g_l^t.
\end{aligned}$$

For $z_l^t-\theta_l^t$, it can be computed similarly. □

We need the following proposition to show the final convergence of Theorem 2.1.

Proposition N.4. *Suppose that the assumptions in Theorem 2.1 hold. For SGD-M (7), we have*

$$\begin{aligned}
&\mathbb{E}\left[\ell\left(z^{t+1}\right)\right] \\
\leq&\mathbb{E}\left[\ell\left(z^{t}\right)\right]+\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2+\sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 \\
&+\sum_{l=1}^{L}(-\eta_l+\frac{\eta_l}{2\rho_0}+\frac{\gamma\eta_l^2}{2})\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2+2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right]\right) \\
&+2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2\gamma^2(\frac{1-\beta_l^t}{1-\beta_l})^2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta_l^{t-i}\nabla_l\ell(\theta^i)-\nabla_l\ell(\theta^t)\right\|^2\right]\right)
\end{aligned} \tag{16}$$

where the auxiliary sequence z_l^t is defined by (15).

Proof. By Lemma N.3 we have that

$$z_l^{t+1}-z_l^t=-\eta_l g_l^t$$

and

$$z_l^t-\theta_l^t=-\frac{\beta_l}{1-\beta_l}\eta_l m_l^{t-1},$$

for all $l=1,2,...,L$.

Now using the Lipschitz smooth of ℓ, we get

$$\begin{aligned}
\mathbb{E}_{\xi_t}\left[\ell\left(z^{t+1}\right)\right]&\leq\ell\left(z^t\right)+\mathbb{E}_{\xi_t}\left[\left\langle\nabla\ell\left(z^t\right),z^{t+1}-z^t\right\rangle\right]+\frac{L}{2}\mathbb{E}_{\xi_t}\left[\left\|z^{t+1}-z^t\right\|^2\right] \\
&=\ell\left(z^t\right)+\sum_{l=1}^{L}\mathbb{E}_{\xi_t}\left[\left\langle\nabla_{\theta_l}\ell\left(z^t\right),-\eta_l g_l^t\right\rangle\right]+\sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\mathbb{E}_{\xi_t}\left[\left\|g_l^t\right\|^2\right] \\
&=\ell\left(z^t\right)+\sum_{l=1}^{L}\left[\left\langle\nabla_{\theta_l}\ell\left(z^t\right),-\eta_l\nabla_l\ell(\theta^t)\right\rangle\right]+\sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\mathbb{E}_{\xi_t}\left[\left\|g_l^t\right\|^2\right]
\end{aligned}$$

where we use the unbiasedness of the gradient estimator in the last line. Now we bound the second term as follows:

$$\begin{aligned}
&\mathbb{E}\left[\left\langle\nabla_{\theta_l}\ell\left(z^t\right),-\eta_l\nabla_l\ell(\theta^t)\right\rangle\right] \\
=&\mathbb{E}\left[\left\langle\nabla_{\theta_l}\ell\left(z^t\right)-\nabla_l\ell(\theta^t),-\eta_l\nabla_l\ell(\theta^t)\right\rangle\right]-\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 \\
\leq&\eta_l\frac{\rho_0}{2}\gamma^2\mathbb{E}\|z^t-\theta^t\|^2+(\frac{\eta_l}{2\rho_0}-\eta_l)\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 \\
=&\eta_l\frac{\rho_0}{2}\sum_l\left(\eta_l^2\mathbb{E}\left[\left(\frac{\beta_l}{1-\beta_l}\right)^2\gamma^2\|m_l^{t-1}\|^2\right]\right)+(\frac{\eta_l}{2\rho_0}-\eta_l)\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2
\end{aligned}$$

where we use Cauchy-Schwarz inequality and ρ_0 is a positive constant to be determined.

Therefore, we get

$$\begin{aligned}
\mathbb{E}_{\xi_t}\left[\ell\left(z^{t+1}\right)\right]\leq&\ell\left(z^t\right)+\frac{\rho_0}{2}\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\mathbb{E}\|m_l^{t-1}\|^2\right) \\
&+\sum_{l=1}^{L}(\frac{\eta_l}{2\rho_0}-\eta_l)\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2+\sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\mathbb{E}_{\xi_t}\left[\left\|g_l^t\right\|^2\right]
\end{aligned} \tag{17}$$

Now from Lemma N.1 we have

$$\mathbb{E}\left[\left\|m_l^{t-1}\right\|^2\right] \leq 2\mathbb{E}\left[\left\|m_l^{t-1} - (1-\beta_l)\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right] + 2\mathbb{E}\left[\left\|(1-\beta_l)\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right]$$
$$\leq 2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 + 2\mathbb{E}\left[\left\|(1-\beta_l)\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right]$$

and

$$\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^{t-1}}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right] \leq 2\mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right] + 2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^{t-1}}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i) - \nabla_l\ell(\theta^t)\right\|^2\right],$$
$$\mathbb{E}\left[\left\|g_l^t\right\|^2\right] \leq \sigma_l^2 + \mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right].$$

plug these into (17) we get:

$$\begin{aligned}
&\mathbb{E}_{\xi_t}\left[\ell\left(z^{t+1}\right)\right] \\
\leq&\ell\left(z^t\right) + \rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2\right) \\
&+ \rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\mathbb{E}\left[\left\|(1-\beta_l)\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right]\right) \\
&+ \sum_{l=1}^L(\frac{\eta_l}{2\rho_0} - \eta_l)\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 + \sum_{l=1}^L\frac{\gamma\eta_l^2}{2}\left(\sigma_l^2 + \mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right]\right) \\
=&\ell\left(z^t\right) + \rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^L\frac{\gamma\eta_l^2}{2}\sigma_l^2 \\
&+ \sum_{l=1}^L(\frac{\eta_l}{2\rho_0} - \eta_l + \frac{\gamma\eta_l^2}{2})\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 \\
&+ \rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^{t-1}}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i)\right\|^2\right]\right) \\
\leq&\ell\left(z^t\right) + \rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^L\frac{\gamma\eta_l^2}{2}\sigma_l^2 \\
&+ \sum_{l=1}^L(-\eta_l + \frac{\eta_l}{2\rho_0} + \frac{\gamma\eta_l^2}{2})\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 + 2\rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right]\right) \\
&+ 2\rho_0\left(\sum_{l=1}^L\eta_l\right)\sum_{l=1}^L\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^{t-1}}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i) - \nabla_l\ell(\theta^t)\right\|^2\right]\right)
\end{aligned}$$

Now the last term above can be replaced since

$$\begin{aligned}
\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^t\beta_l^{t-i}\nabla_l\ell(\theta^i) - \nabla_l\ell(\theta^t)\right\|^2\right] =&\mathbb{E}\left[\left\|\frac{\beta_l(1-\beta_l)}{1-\beta_l^t}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i) - \frac{1-\beta_l^{t-1}}{1-\beta_l^t}\beta_l\nabla_l\ell(\theta^t)\right\|^2\right] \\
=&\beta_l^2(\frac{1-\beta_l^{t-1}}{1-\beta_l^t})^2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^{t-1}}\sum_{i=1}^{t-1}\beta_l^{t-1-i}\nabla_l\ell(\theta^i) - \nabla_l\ell(\theta^t)\right\|^2\right]
\end{aligned}$$

□

Now we return to the proof of Theorem 2.1.

Proof of Theorem 2.1. By Proposition N.4 and Lemma N.2, we have

$$\begin{aligned}
&\mathbb{E}\left[\ell\left(z^{t+1}\right)\right] \\
\leq&\mathbb{E}\left[\ell\left(z^{t}\right)\right] + \rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 \\
&+\sum_{l=1}^{L}(-\eta_l + \frac{\eta_l}{2\rho_0} + \frac{\gamma\eta_l^2}{2})\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 + 2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right]\right) \\
&+2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2\gamma^2(\frac{1-\beta_l^t}{1-\beta_l})^2\sum_{i=1}^{t-1}a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right]\right)
\end{aligned} \tag{18}$$

Now define the potential function:

$$\phi^t = \ell\left(z^t\right) - \ell^* + \sum_{i=1}^{t-1}c_i\left\|\theta^{t+1-i}-\theta^{t-i}\right\|^2 \tag{19}$$

where c_i are constants to be determined.

From (18) we get

$$\begin{aligned}
&\mathbb{E}\left[\phi^{t+1}\right] - \mathbb{E}\left[\phi^t\right] \\
\leq&\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 \\
&+\sum_{l=1}^{L}(-\eta_l + \frac{\eta_l}{2\rho_0} + \frac{\gamma\eta_l^2}{2})\mathbb{E}\|\nabla_l\ell(\theta^t)\|^2 + 2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\nabla_l\ell(\theta^t)\right\|^2\right]\right) \\
&+2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2\gamma^2(\frac{1-\beta_l^t}{1-\beta_l})^2\sum_{i=1}^{t-1}a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right]\right) \\
&+c_1\mathbb{E}\left\|\theta^{t+1}-\theta^t\right\|^2 + \sum_{i=1}^{t-1}(c_{i+1}-c_i)\mathbb{E}\left[\left\|\theta^{t+1-i}-\theta^{t-i}\right\|^2\right]
\end{aligned} \tag{20}$$

For the term $c_1\mathbb{E}\left\|\theta^{t+1}-\theta^t\right\|^2$, we can bound it by (denote $\nabla_l^t = \nabla_{\theta_l}\ell(\theta^t)$)

$$\begin{aligned}
&c_1\mathbb{E}\left\|\theta^{t+1}-\theta^t\right\|^2 = \sum_{l=1}^{L}c_1\mathbb{E}\left\|\theta_l^{t+1}-\theta_l^t\right\|^2 = \sum_{l=1}^{L}c_1\eta_l^2\mathbb{E}\left[\left\|m_l^t\right\|^2\right] \\
\leq&2\sum_{l=1}^{L}c_1\mathbb{E}\left[\left\|m_l^t - (1-\beta_l)\sum_{i=1}^{t}c_1\beta_l^{t-i}\nabla_l^i\right\|^2\right] + 2\sum_{l=1}^{L}\mathbb{E}\left[\left\|(1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}\nabla_l^i\right\|^2\right] \\
\leq&2\sum_{l=1}^{L}c_1\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 + 2\sum_{l=1}^{L}c_1\mathbb{E}\left[\left\|(1-\beta_l)\sum_{i=1}^{t}\beta_l^{t-i}\nabla_l^i\right\|^2\right] \\
\leq&\sum_{l=1}^{L}c_1\eta_l^2\left(2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 + 4\mathbb{E}\left[\left\|\nabla_l^t\right\|^2\right]\left(1-\beta_l^t\right)^2\right) + 4\sum_{l=1}^{L}c_1(1-\beta_l^t)^2\eta_l^2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta^{t-i}\nabla_l^i - \nabla_l^t\right\|^2\right] \\
\leq&\sum_{l=1}^{L}c_1\eta_l^2\left(2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 + 4\mathbb{E}\left[\left\|\nabla_l^t\right\|^2\right]\right) + 4\sum_{l=1}^{L}c_1(1-\beta_l^t)^2\eta_l^2\sum_{i=1}^{t-1}a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right]
\end{aligned} \tag{21}$$

where for the last three inequalities we use (12) and

$$\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta_l^{t-i}\nabla_l^i\right\|^2\right] \le 2\mathbb{E}\left[\left\|\nabla_l^t\right\|^2\right] + 2\mathbb{E}\left[\left\|\frac{1-\beta_l}{1-\beta_l^t}\sum_{i=1}^{t}\beta_l^{t-i}\nabla_l^i - \nabla_l^t\right\|^2\right],$$

and (13), respectively.

Now plugging this back to (20) we get:

$$\begin{aligned}
&\mathbb{E}\left[\phi^{t+1}\right] - \mathbb{E}\left[\phi^t\right] \\
\le&\rho_0 s\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 + \sum_{l=1}^{L}2c_1\eta_l^2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 \\
&+\sum_{l=1}^{L}(-\eta_l + \frac{\eta_l}{2\rho_0} + \frac{\gamma\eta_l^2}{2} + 4c_1\eta_l^2)\mathbb{E}\|\nabla_l^t\|^2 + 2\rho_0\left(\sum_{l=1}^{L}\eta_l\right)\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\mathbb{E}\left[\left\|\nabla_l^t\right\|^2\right]\right) \\
&+4\sum_{i=1}^{t-1}\sum_{l=1}^{L}c_1(1-\beta_l^t)^2\eta_l^2 a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right] \\
&+2\rho_0 s\sum_{i=1}^{t-1}\sum_{l=1}^{L}\left(\eta_l^2\gamma^2(\frac{1-\beta_l^t}{1-\beta_l})^2 a_{l,t,i}\mathbb{E}\left[\left\|\theta^{i+1}-\theta^i\right\|^2\right]\right) \\
&+\sum_{i=1}^{t-1}(c_{i+1}-c_i)\mathbb{E}\left[\left\|\theta^{t+1-i}-\theta^{t-i}\right\|^2\right]
\end{aligned} \tag{22}$$

where we denote $s := \sum_{l=1}^{L}\eta_l$.

To make the last three lines of above non-positive, we could take

$$c_{i+1} \le c_i - \sum_{l=1}^{L}\left(4c_1(1-\beta_l^t)^2\eta_l^2 + 2\rho_0 s\eta_l^2\gamma^2(\frac{1-\beta_l^t}{1-\beta_l})^2\right)a_{l,t,t-i}.$$

We can take (since $1-\beta_l^t < 1$)

$$c_{i+1} = c_i - \sum_{l=1}^{L}\left(4c_1\eta_l^2 + 2\rho_0 s\eta_l^2\gamma^2(\frac{1}{1-\beta_l})^2\right)\gamma^2\beta_l^i\left(i + \frac{\beta_l}{1-\beta_l}\right)$$

also we can take c_1 such that

$$\begin{aligned}
c_1 =& \sum_{l=1}^{L}\sum_{i=1}^{\infty}\left(4c_1\eta_l^2 + 2\rho_0 s\eta_l^2\gamma^2(\frac{1}{1-\beta_l})^2\right)\gamma^2\beta_l^i\left(i + \frac{\beta_l}{1-\beta_l}\right) \\
=& \sum_{l=1}^{L}\left(4c_1\eta_l^2 + 2\rho_0 s\eta_l^2\gamma^2(\frac{1}{1-\beta_l})^2\right)\gamma^2\left(\sum_{i=1}^{\infty}i\beta_l^i + \frac{\beta_l}{1-\beta_l}\sum_{i=1}^{\infty}\beta_l^i\right) \\
=& \sum_{l=1}^{L}\left(4c_1\eta_l^2 + 2\rho_0 s\eta_l^2\gamma^2(\frac{1}{1-\beta_l})^2\right)\frac{\gamma^2\beta_l(1+\beta_l)}{(1-\beta_l)^2}
\end{aligned}$$

i.e.

$$c_1 = \frac{\sum_{l=1}^{L}2\rho_0 s\eta_l^2\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4}}{1 - 4\sum_{l=1}^{L}\eta_l^2\frac{\gamma^2\beta_l(1+\beta_l)}{(1-\beta_l)^2}}. \tag{23}$$

Note that we can give an upper bound for c_1 by requiring the denominator $\ge 1/2$, i.e.

$$\eta_l \le \frac{1-\beta_l}{\gamma\sqrt{8\beta_l(1+\beta_l)L}},\ \forall l = 1, 2, ..., L, \tag{24}$$

consequently

$$\sum_{l=1}^{L} 2\rho_0 s\eta_l^2\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4} \le c_1 \le \sum_{l=1}^{L} 4\rho_0 s\eta_l^2\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4}. \tag{25}$$

Now we have (since the last three terms of (22) sum to negative)

$$\begin{aligned}
&\mathbb{E}\left[\phi^{t+1}\right] - \mathbb{E}\left[\phi^{t}\right] \\
\le&\rho_0 s\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 + \sum_{l=1}^{L}2c_1\eta_l^2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2 \\
&+\sum_{l=1}^{L}\left(-\eta_l + \frac{\eta_l}{2\rho_0} + \frac{\gamma\eta_l^2}{2} + 4c_1\eta_l^2 + 2\rho_0 s\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2\right)\mathbb{E}\|\nabla_l^t\|^2
\end{aligned} \tag{26}$$

Now take $\rho_0 = 2$, and take η_l such that

$$-\frac{3}{4}\eta_l + \frac{\gamma\eta_l^2}{2} + 4c_1\eta_l^2 + 4s\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2(1-\beta_l^{t-1})^2 \le -\frac{\eta_l}{2}, \tag{27}$$

the coefficient of $\mathbb{E}\|\nabla_l^t\|$ can be greatly simplified. Note that we can guarantee (27) if

$$\eta_l \le \min\left\{\frac{1}{8\gamma}, \frac{1}{64\sum_{l=1}^{L}\rho_0 s\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4}}, \frac{1-\beta_l}{4\gamma}\sqrt[3]{\frac{1-\beta_l}{L\beta_l(1+\beta_l)}}\right\} \tag{28}$$

for all $l = 1, 2, ..., L$. Here each term in the right hand side of (28) is bounding each term on the left hand side of (27).

We get

$$\begin{aligned}
&\sum_{l=1}^{L}\frac{\eta_l}{2}\mathbb{E}\|\nabla_l^t\|^2 \le \mathbb{E}\left[\phi^{t}\right] - \mathbb{E}\left[\phi^{t+1}\right] \\
&+ 2s\sum_{l=1}^{L}\left(\eta_l^2(\frac{\beta_l}{1-\beta_l})^2\gamma^2\frac{1-\beta_l}{1+\beta_l}\right)\sigma_l^2 + \sum_{l=1}^{L}\frac{\gamma\eta_l^2}{2}\sigma_l^2 + \sum_{l=1}^{L}2c_1\eta_l^2\frac{1-\beta_l}{1+\beta_l}\sigma_l^2.
\end{aligned} \tag{29}$$

Note that we have defined $s := \sum_{l=1}^{L}\eta_l$. We can upper bound it by $s \le L/\sqrt{\gamma T}$ if we assume $\eta_l \le 1/\sqrt{\gamma T}$. Combining (24) and (28) we get:

$$\eta_l \le \min\left\{\frac{1}{8\gamma}, \frac{1}{8\gamma L}(\frac{1-\beta_l}{\beta_l})^2, \frac{1-\beta_l}{\gamma\sqrt{8\beta_l(1+\beta_l)}}, \frac{1-\beta_l}{4\gamma}\sqrt[3]{\frac{1-\beta_l}{L\beta_l(1+\beta_l)}}, \frac{1}{\sqrt{\gamma T}}\right\}, \tag{30}$$

which is satisfied if (since $\beta_l(1+\beta_l) \le 2$)

$$\eta_l = \min\left\{\frac{1}{8\gamma}, \frac{1}{64\sum_{l=1}^{L}\rho_0 s\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4}}, \frac{1-\beta_l}{4\gamma}, \frac{1-\beta_l}{4\gamma}\sqrt[3]{\frac{1-\beta_l}{2L}}, \frac{1}{\sqrt{\gamma T}}\right\}. \tag{31}$$

By taking the product of two of the terms in the RHS of the above relation, we have the following upper bound on η_l^2:

$$\eta_l^2 \le \frac{1}{8\gamma L}(\frac{1-\beta_l}{\beta_l})^2\frac{1}{\sqrt{\gamma T}}. \tag{32}$$

Plugging (32) and (25) into (29), we obtain:

$$\begin{aligned}
\sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l^t\|^2 \le&2(\mathbb{E}\left[\phi^{t}\right] - \mathbb{E}\left[\phi^{t+1}\right]) \\
&+\sum_{l=1}^{L}\left(\frac{1-\beta_l}{1+\beta_l}\frac{1}{4\gamma T} + \frac{1}{2T} + \frac{1-\beta_l}{\beta_l^3}\frac{\sqrt{\gamma}}{4L^2T^{3/2}}\right)\sigma_l^2.
\end{aligned} \tag{33}$$

Now since $\beta_l \leq 1-\delta$, we have that η_l is lower bounded

$$\begin{aligned}\eta_l &= \min\left\{\frac{1}{8\gamma}, \frac{1}{64\sum_{l=1}^{L}\rho_0 s\gamma^4\beta_l\frac{1+\beta_l}{(1-\beta_l)^4}}, \frac{1-\beta_l}{4\gamma}, \frac{1-\beta_l}{4\gamma}\sqrt[3]{\frac{1-\beta_l}{2L}}\right\} \\ &\geq \min\left\{\frac{1}{8\gamma}, \frac{\delta^4}{64L\rho_0 s\gamma^4}, \frac{\delta}{4\gamma}, \frac{\delta}{4\gamma}\sqrt[3]{\frac{\delta}{2L}}, \frac{1}{\sqrt{\gamma T}}\right\} =: \eta \end{aligned} \tag{34}$$

we obtain:

$$\begin{aligned}\sum_{l=1}^{L}\mathbb{E}\|\nabla_l^t\|^2 \leq& \frac{2(\mathbb{E}\left[\phi^t\right] - \mathbb{E}\left[\phi^{t+1}\right])}{\eta} \\ &+ \frac{1}{\eta}\sum_{l=1}^{L}\left(\frac{1-\beta_l}{1+\beta_l}\frac{1}{4\gamma T} + \frac{1}{2T} + \frac{1-\beta_l}{\beta_l^3}\frac{\sqrt{\gamma}}{4L^2T^{3/2}}\right)\sigma_l^2.\end{aligned} \tag{35}$$

Now by telescoping sum of (33) for $t = 1, ..., T$, also notice that $1/\eta = \Theta(\sqrt{\gamma T}L\gamma/\delta^2)$ as T grows, we get our final result of

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{l=1}^{L}\mathbb{E}\|\nabla_l^t\|^2 \leq \frac{2L\gamma^{3/2}\mathbb{E}\Delta_1}{\delta^2\sqrt{T}} + \sum_{l=1}^{L}\left(\frac{1-\beta_l}{1+\beta_l}\frac{L\sqrt{\gamma}}{4\sqrt{T}} + \frac{L\gamma^{3/2}}{2\sqrt{T}} + \frac{1-\beta_l}{\beta_l^3}\frac{\gamma^2}{4LT}\right)\frac{\sigma_l^2}{\delta^2}. \tag{36}$$

This completes the proof. □

O. Proofs for Section 3

In this section, we conduct the proof for Theorem 3.1. The proof follows Riabinin et al. (2025), with changes to adopt to our algorithms.

First, let us recall the algorithm:

$$\begin{aligned}&\text{(for each layer } l)\ m_l^t = \beta_l m_l^{t-1} + (1-\beta_l)g_l^t \\ &\text{(for each layer } l)\ \theta_l^{t+1} = \theta_l^t - \eta_l\mathcal{C}(m_l^t).\end{aligned} \tag{37}$$

And the assumptions:

Assumption O.1. We assume that

- $\ell(\theta)$ in (1) is lower bounded by ℓ^*;
- each layer l is L_l-smooth in matrix $1\to 2$ norm: $\|\nabla_{\theta_l}\ell(\theta) - \nabla_{\theta_l}\ell(\vartheta)\|_{2\to\infty} \leq L_l\|\theta - \vartheta\|_{1\to 2}$;
- the stochastic gradient is unbiased $\mathbb{E}_{\xi_t}\nabla_{\theta_l}\ell(\theta^t;\xi_t) = \nabla_{\theta_l}\ell(\theta^t)$ and with bounded variance:

$$\mathbb{E}_{\xi_t}\|\left(\nabla_{\theta_l}\ell(\theta^t;\xi_t) - \nabla_{\theta_l}\ell(\theta^t)\right)_{:,j}\|_{2\to\infty}^2 \leq \nu_{l,j}^2$$

  for all $l = 1, ..., L$ and $t = 0, ..., T-1$ and all the columns j. Also denote $\bar{\sigma}_l := \left(\sum_{j=1}^{n_l}\nu_{l,j}^2\right)^{1/2}$.

Now we proceed to the proof.

Proof of Theorem 3.1. Let

$$\Delta_l^t := \theta_l^{t+1} - \theta_l^t.$$

By the layerwise Lipschitz smoothness assumption, for every $t \geq 0$,

$$\ell(\theta^{t+1}) \leq \ell(\theta^t) + \sum_{l=1}^{L}\left[\langle\nabla_l\ell(\theta^t), \Delta_l^t\rangle_F + \frac{L_l}{2}\|\Delta_l^t\|_{1\to 2}^2\right].$$

For each layer l, the update is

$$\theta_l^{t+1} = \theta_l^t - \eta_l \mathcal{C}(m_l^t).$$

From (Bernstein & Newhouse, 2024b), we have the following identity for any matrix G[7]:

$$\arg\min_{\|\Delta\|_{1\to 2}\le 1} \langle G, \Delta\rangle = -\left[\frac{\mathrm{col}_1(G)}{\|\mathrm{col}_1(G)\|_2}, \ldots, \frac{\mathrm{col}_n(G)}{\|\mathrm{col}_n(G)\|_2}\right].$$

Therefore for each layer l, the update is equivalently

$$\theta_l^{t+1} = \arg\min_{\|U-\theta_l^t\|_{1\to 2}\le \eta_l} \langle m_l^t, U\rangle_F.$$

Hence Δ_l^t belongs to the radius-η_l ball in the $\|\cdot\|_{1\to 2}$ norm, so

$$\|\Delta_l^t\|_{1\to 2} \le \eta_l.$$

Moreover, by the definition of the dual norm, the minimum of the linear functional over this ball satisfies

$$\langle m_l^t, \Delta_l^t\rangle_F = -\eta_l \|m_l^t\|_{*,l},$$

where $\|\cdot\|_{*,l}$ denotes the dual norm of $\|\cdot\|_{1\to 2}$, namely $\|\cdot\|_{2\to\infty}$.

Define the tracking error

$$e_l^t := m_l^t - \nabla_l \ell(\theta^t).$$

Then

$$\langle \nabla_l \ell(\theta^t), \Delta_l^t\rangle_F = \langle m_l^t, \Delta_l^t\rangle_F - \langle e_l^t, \Delta_l^t\rangle_F.$$

By Hölder's inequality for dual norms,

$$\|\langle e_l^t, \Delta_l^t\rangle_F\| \le \|e_l^t\|_{*,l}\|\Delta_l^t\|_{1\to 2} \le \eta_l \|e_l^t\|_{*,l}.$$

Therefore,

$$\langle \nabla_l \ell(\theta^t), \Delta_l^t\rangle_F \le -\eta_l \|m_l^t\|_{*,l} + \eta_l \|e_l^t\|_{*,l}.$$

Using the reverse triangle inequality,

$$\|m_l^t\|_{*,l} = \|\nabla_l \ell(\theta^t) + e_l^t\|_{*,l} \ge \|\nabla_l \ell(\theta^t)\|_{*,l} - \|e_l^t\|_{*,l},$$

we further obtain

$$\langle \nabla_l \ell(\theta^t), \Delta_l^t\rangle_F \le -\eta_l \|\nabla_l \ell(\theta^t)\|_{*,l} + 2\eta_l \|e_l^t\|_{*,l}.$$

Substituting the above estimates into the descent inequality gives

$$\ell(\theta^{t+1}) \le \ell(\theta^t) - \sum_{l=1}^L \eta_l \|\nabla_l \ell(\theta^t)\|_{*,l} + 2\sum_{l=1}^L \eta_l \|e_l^t\|_{*,l} + \sum_{l=1}^L \frac{L_l}{2}\eta_l^2.$$

Taking expectation yields

$$\mathbb{E}\ell(\theta^{t+1}) \le \mathbb{E}\ell(\theta^t) - \sum_{l=1}^L \eta_l, \mathbb{E}\|\nabla_l \ell(\theta^t)\|_{*,l} + 2\sum_{l=1}^L \eta_l \mathbb{E}\|e_l^t\|_{*,l} + \sum_{l=1}^L \frac{L_l}{2}\eta_l^2.$$

It remains to bound $\mathbb{E}\|e_l^t\|_{*,l}$. From the momentum recursion

$$m_l^t = \beta_l m_l^{t-1} + (1-\beta_l) g_l^t = \beta_l m_l^{t-1} + (1-\beta_l)\big(\nabla_l \ell(\theta^t) + \zeta_l^t\big),$$

we get

$$e_l^t = m_l^t - \nabla_l \ell(\theta^t) = \beta_l\big(m_l^{t-1} - \nabla_l \ell(\theta^{t-1})\big) + \beta_l\big(\nabla_l \ell(\theta^{t-1}) - \nabla_l \ell(\theta^t)\big) + (1-\beta_l)\zeta_l^t.$$

[7] without loss of generality, we could assume none of the columns of G are zero

That is,

$$e_l^t = \beta_l e_l^{t-1} + \beta_l \delta_l^t + (1-\beta_l)\zeta_l^t, \qquad \delta_l^t := \nabla_l \ell(\theta^{t-1}) - \nabla_l \ell(\theta^t).$$

Since $m_l^0 = \nabla_l \ell(\theta^0)$, we have $e_l^0 = 0$. Unrolling the recursion gives

$$e_l^t = \beta_l \sum_{s=1}^{t} \beta_l^{t-s} \delta_l^s + (1-\beta_l) \sum_{s=1}^{t} \beta_l^{t-s} \zeta_l^s.$$

We bound the two terms separately.

For the first drifting term, by the layerwise smoothness assumption and the bound $\|\Delta_l^{s-1}\|_{1\to 2} \le \eta_l$,

$$\|\delta_l^s\|_{*,l} = \|\nabla_l \ell(\theta^{s-1}) - \nabla_l \ell(\theta^s)\|_{*,l} \le L_l \|\theta_l^{s-1} - \theta_l^s\|_{1\to 2} = L_l \|\Delta_l^{s-1}\|_{1\to 2} \le L_l \eta_l.$$

Therefore,

$$\left\| \beta_l \sum_{s=1}^{t} \beta_l^{t-s} \delta_l^s \right\|_{*,l} \le \beta_l \sum_{s=1}^{t} \beta_l^{t-s} \|\delta_l^s\|_{*,l} \le \beta_l \sum_{s=1}^{t} \beta_l^{t-s} L_l \eta_l \le \frac{\beta_l L_l \eta_l}{1-\beta_l}.$$

Now consider the noise term

$$N_l^t := (1-\beta_l) \sum_{s=1}^{t} \beta_l^{t-s} \zeta_l^s.$$

Since the layer norm is $\|\cdot\|_{1\to 2}$, we have

$$\|N_l^t\|_{1\to 2} = \max_{1\le j\le n_l} \|(N_l^t) * :, j\|_2.$$

Using $\mathbb{E}X \le (\mathbb{E}X^2)^{1/2}$ for nonnegative X, and then $\max_j a_j \le \sum_j a_j$ for nonnegative numbers a_j,

$$\mathbb{E}\|N_l^t\|_{1\to 2} \le \left(\mathbb{E}\|N_l^t\|_{1\to 2}^2\right)^{1/2} = \left(\mathbb{E} \max_{1\le j\le n_l} \|(N_l^t) * :, j\|_2^2\right)^{1/2} \le \left(\sum_{j=1}^{n_l} \mathbb{E}\|(N_l^t) * :, j\|_2^2\right)^{1/2}.$$

Fix a column $j \in 1, \ldots, n_l$. Since

$$(N_l^t)_{:,j} = (1-\beta_l) \sum_{s=1}^{t} \beta_l^{t-s} (\zeta_l^s)_{:,j},$$

we obtain

$$\mathbb{E}\|(N_l^t) * :, j\|_2^2 = (1-\beta_l)^2 \mathbb{E} \left\| \sum_{s=1}^{t} \beta_l^{t-s} (\zeta_l^s)_{:,j} \right\|_2^2.$$

Expanding the Euclidean square,

$$\mathbb{E}\|(N_l^t) * :, j\|_2^2 = (1-\beta_l)^2 \sum_{s,r=1}^{t} \beta_l^{t-s} \beta_l^{t-r}, \mathbb{E}\left\langle (\zeta_l^s)_{:,j}, (\zeta_l^r)_{:,j} \right\rangle.$$

For $s \ne r$, the cross terms vanish by unbiasedness,

$$\mathbb{E}\left\langle (\zeta_l^s)_{:,j}, (\zeta_l^r)_{:,j} \right\rangle = 0$$

Hence only the diagonal terms remain:

$$\mathbb{E}\|(N_l^t)_{:,j}\|_2^2 \le (1-\beta_l)^2 \sum_{s=1}^{t} \beta_l^{2(t-s)} \mathbb{E}\|(\zeta_l^s)_{:,j}\|_2^2.$$

Using the column-wise second-moment bound,

$$\mathbb{E}\|(\zeta_l^s)_{:,j}\|_2^2 = \mathbb{E}\Big[\mathbb{E}\big[\|(\zeta_l^s)_{:,j}\|_2^2 \mid \mathcal{F}_s\big]\Big] \le \nu_{l,j}^2,$$

so

$$\mathbb{E}\|(N_l^t)_{:,j}\|_2^2 \le (1-\beta_l)^2\nu_{l,j}^2 \sum_{q=0}^{t-1}\beta_l^{2q}.$$

Since

$$\sum_{q=0}^{t-1}\beta_l^{2q} \le \frac{1}{1-\beta_l^2},$$

we obtain

$$\mathbb{E}\|(N_l^t)_{:,j}\|_2^2 \le (1-\beta_l)^2\nu_{l,j}^2\frac{1-\beta_l}{1+\beta_l}.$$

Summing over all columns j gives

$$\mathbb{E}\|N_l^t\|_{1\to 2} \le \left(\sum_{j=1}^{n_l}\nu_{l,j}^2\frac{1-\beta_l}{1+\beta_l}\right)^{1/2} = \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}.$$

Combining the drift bound and the noise bound, and using the triangle inequality, we get

$$\mathbb{E}\|e_l^t\|_{*,l} \le \frac{\beta_l L_l\eta_l}{1-\beta_l} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}.$$

Substituting this estimate into the expected descent inequality yields

$$\mathbb{E}\ell(\theta^{t+1}) \le \mathbb{E}\ell(\theta^t) - \sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l} + \sum_{l=1}^{L}\left[2\eta_l\left(\frac{\beta_l L_l\eta_l}{1-\beta_l} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}\right) + \frac{L_l}{2}\eta_l^2\right].$$

Summing this inequality over $t = 0, 1, \ldots, T-1$ and telescoping, we have

$$\sum_{t=0}^{T-1}\sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l} \le \ell(\theta^0) - \mathbb{E}\ell(\theta^T) + T\sum_{l=1}^{L}\left[2\eta_l\left(\frac{\beta_l L_l\eta_l}{1-\beta_l} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}\right) + \frac{L_l}{2}\eta_l^2\right].$$

Since $\ell(\theta^T) \ge \ell_{\inf}$,

$$\sum_{t=0}^{T-1}\sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l} \le \Delta_0 + T\sum_{l=1}^{L}\left[2\eta_l\left(\frac{\beta_l L_l\eta_l}{1-\beta_l} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}\right) + \frac{L_l}{2}\eta_l^2\right].$$

Therefore,

$$T\min_{0\le t\le T-1}\sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l} \le \sum_{t=0}^{T-1}\sum_{l=1}^{L}\eta_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l},$$

and hence

$$\min_{0\le t\le T-1}\sum_{l=1}^{L}\eta_l, \mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l} \le \frac{\Delta_0}{T} + \sum_{l=1}^{L}\left[2\eta_l\left(\frac{\beta_l L_l\eta_l}{1-\beta_l} + \bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}\right) + \frac{L_l}{2}\eta_l^2\right].$$

Finally, substitute $\eta_l = a_l T^{-3/4}$. Since

$$\sum_{l=1}^{L}\eta_l = T^{-3/4}\sum_{l=1}^{L}a_l = AT^{-3/4},$$

dividing both sides by $T^{-3/4}$ yields

$$\min_{0\leq t\leq T-1}\sum_{l=1}^{L}a_l\mathbb{E}\|\nabla_l\ell(\theta^t)\|_{*,l}\leq\frac{\Delta_0}{T^{1/4}}+\sum_{l=1}^{L}\left[\frac{L_l a_l^2}{2}T^{-3/4}+2a_l\left(\frac{\beta_l L_l a_l}{1-\beta_l}T^{-1/2}+\bar{\sigma}_l\sqrt{\frac{1-\beta_l}{1+\beta_l}}T^{-1/4}\right)\right].$$

This proves the theorem. □